
Deep Reinforcement Learning

Master Summary

Stefan Eder

Based on Lecture *Deep Reinforcement Learning* SS26, JKU Linz

How to use this document. This is a structured summary built around *running text*. Explanations, derivations, and logical flow between ideas live in the prose — colored boxes are reserved for the few things that must stand out at a glance.

- **Running text** — read for understanding, derivations, and context.
- **Colored boxes** — scan for key definitions, theorems, examples, and exam notes.

Boxes lose their impact when overused. When in doubt, write prose.

Contents

1	RL Foundations	1
1.1	What is Reinforcement Learning?	1
1.2	The Agent–Environment Interface	2
1.3	Goals and Returns	3
1.4	Policies and Value Functions	4
1.5	Exploration vs. Exploitation	5
1.6	Multi-Armed Bandits	5
1.6.1	Problem setup	5
1.6.2	Action value estimation	5
1.6.3	Regret	6
1.6.4	Algorithms	6
2	Tabular Value-Based Methods	9
2.1	RL Taxonomy	9
2.2	Bellman Equations	10
2.3	Planning with Dynamic Programming	11
2.3.1	Policy evaluation and improvement	11
2.3.2	Policy iteration and value iteration	11
2.4	Model-Free Prediction	12
2.4.1	Monte Carlo: learn from complete episodes	12
2.4.2	Temporal-difference: learn online, one step at a time	12
2.4.3	Multi-step returns and TD(λ)	13
2.5	Model-Free Control	13
2.5.1	SARSA: on-policy TD control	14
2.5.2	Q-learning: off-policy TD control	14
2.5.3	Maximization bias and Double Q-learning	15
2.5.4	Off-policy learning and importance sampling	15
3	Value Function Approximation	17
3.1	Learning Value Functions by Gradient Descent	17
3.2	Function Approximators	17
3.3	Action-Value Approximation and Control	18
3.4	Batch Learning and Experience Replay	18
3.5	Deep Q-Networks	19
3.6	Beyond DQN: Key Extensions	19
4	Policy-Based Methods	22
4.1	Why Bother With Policies Directly?	22
4.1.1	Continuous action spaces	22
4.1.2	Deterministic policies are not always enough	23
4.1.3	Interlude: POMDPs	23
4.1.4	Trade-offs	24
4.2	Setting Up the Optimization	24

4.2.1	Episodic objective: average return per episode	24
4.2.2	Continuing objective: average reward per step	24
4.2.3	Optimization choices	24
4.3	Computing the Policy Gradient	25
4.3.1	Finite differences	25
4.3.2	The score-function (log-likelihood) trick	25
4.3.3	Example: softmax policy	26
4.4	The Policy Gradient Theorem	26
4.4.1	From theorem to algorithm: REINFORCE	28
4.5	Reducing Variance: Baselines	28
4.5.1	The baseline identity	29
4.5.2	Why a state-value baseline?	29
4.6	Actor-Critic Methods	29
4.6.1	From baseline to critic	30
4.6.2	n-step actor-critic	30
4.6.3	Generalized Advantage Estimation (GAE)	31
4.7	Recap	31
5	Modern Actor-Critic Algorithms	32
5.1	A Unified View of Policy Gradients	32
5.1.1	Bias and variance, again	33
5.1.2	Why γ^t usually disappears	33
5.2	Off-Policy Policy Gradients	33
5.2.1	Importance sampling	33
5.3	Stable Updates: NPG, TRPO, PPO	34
5.3.1	Measuring policy change with KL divergence	34
5.3.2	Natural Policy Gradient (NPG)	34
5.3.3	Trust Region Policy Optimization (TRPO)	35
5.3.4	Proximal Policy Optimization (PPO)	35
5.3.5	NPG, TRPO, PPO at a glance	36
5.4	Continuous Control	36
5.4.1	The Gaussian policy	36
5.4.2	Deterministic Policy Gradient (DPG)	37
5.4.3	Deep Deterministic Policy Gradient (DDPG)	37
5.4.4	TD3	37
5.4.5	Maximum-entropy RL and SAC	38
5.4.6	When to use what	38
5.5	Recap	39
6	Core Challenges in Deep RL	40
6.1	Exploration	40
6.1.1	Why exploration is hard	40
6.1.2	Baseline exploration: random and noise-based	41
6.1.3	Three guiding principles for directed exploration	41
6.1.4	Uncertainty-based exploration	41
6.1.5	Novelty-based exploration: intrinsic motivation	42
6.1.6	Where exploration fits in the pipeline	43
6.2	Credit Assignment	43
6.2.1	Three root causes	43
6.2.2	Time-contiguity methods (depth)	44

6.2.3	Reducing breadth noise with advantages	44
6.2.4	Return decomposition (RUDDER) for very long delays	44
6.2.5	Hindsight Experience Replay (HER) for sparse goals	45
6.2.6	Practical takeaways	45
6.3	State Representation Learning	45
6.3.1	End-to-end RL vs. SRL + RL	45
6.3.2	What makes a representation good	46
6.3.3	A taxonomy of SRL methods	46
6.3.4	Why representation matters in practice	47
6.4	Recap	47

RL Foundations

Reinforcement learning (RL) is the third major paradigm of machine learning alongside supervised and unsupervised learning. In **supervised learning** we learn a mapping from labeled input–output pairs; in **unsupervised learning** we discover structure in unlabeled data. RL is different: an **agent** interacts with an **environment**, receives feedback in the form of a scalar reward signal, and learns to act so as to maximize cumulative reward. The first lecture of this course is a structured recap of the core formalism — the machinery developed here underpins every deep-RL method covered later.

1.1 What is Reinforcement Learning?

Reinforcement learning is a *computational approach of learning to make decisions from interactions*. The agent is not told what the correct action is; it must discover which actions lead to high reward by trying them. This is what separates RL from supervised learning, where target labels are provided.

Intuition & Mental Model

Five defining characteristics of RL:

- **No labeled targets** — only a scalar reward signal, not explicit supervision.
- **Delayed feedback** — reward may arrive long after the action that caused it (the *credit-assignment* problem).
- **Active data collection** — the agent’s actions influence which states are visited next, so the data distribution is non-stationary and policy-dependent.
- **Sequential interactions** — decisions at time t affect states at $t + 1, t + 2, \dots$, creating long-term dependencies.
- **Goal-directed** — the objective is to maximize cumulative reward, not any single step’s outcome.

The overarching assumption that makes the RL framework broadly applicable is the *reward hypothesis*.

Reward Hypothesis

All goals and purposes of an agent can be described as the maximization of the *expected cumulative reward*.

This is a conjecture, not a theorem — but it is a remarkably useful design principle for defining objectives across robotics, games, language models, and beyond.

1.2 The Agent–Environment Interface

At each discrete time step $t = 0, 1, 2, \dots$ the agent observes the current state S_t , selects an action A_t , and the environment responds with a scalar reward R_{t+1} and the next state S_{t+1} . The agent executes A_t and receives observation O_t and reward R_t ; the environment receives A_t and emits O_{t+1} and R_{t+1} .

Trajectory

The agent observes state $S_t \in \mathcal{S}$ and selects action $A_t \in \mathcal{A}(S_t)$. The environment responds with reward $R_{t+1} \in \mathbb{R}$ and next state S_{t+1} . The resulting sequence

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots$$

is called a *trajectory* (or history).

A useful tool throughout RL is the conditional expectation. Recall $\mathbb{E}[X] = \sum_i x_i \Pr\{X = x_i\}$ and $\mathbb{E}[X | Y = y] = \sum_i x_i \Pr\{X = x_i | Y = y\}$. The *law of total expectation*, $\mathbb{E}[X | Y] = \sum_z \Pr\{Z = z | Y\} \mathbb{E}[X | Y, Z = z]$, is used repeatedly when expanding value functions over actions and next states.

To formalize the environment we use the Markov Decision Process framework. The slides present two equivalent definitions; the first is standard in theoretical work, the second separates transition dynamics from expected reward.

Markov Decision Process

A **Markov Decision Process (MDP)** is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma \rangle$, where

- $\mathcal{S}$ is the finite set of **states**,
- $\mathcal{A}$ is the finite set of **actions**,
- $\mathcal{P}$ specifies the **joint transition probabilities** (Markov):

$$p(s', r | s, a) = \Pr(S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a),$$

- $\gamma \in [0, 1]$ is the **discount factor**.

Alternative formulation. Separate state transitions from expected reward:

$$p(s' | s, a) = \sum_r p(s', r | s, a),$$

$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_r r \sum_{s'} p(s', r | s, a).$$

Both formulations define the same dynamics; use whichever is more convenient.

The crucial assumption embedded in the MDP definition is the *Markov property*: the next state and reward depend on history only through the current state–action pair.

Markov Property

A state signal is *Markov* if

$$\begin{aligned} \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a, \text{past}) \\ = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a). \end{aligned}$$

“The future is independent of the past given the present.”

The state must capture all information relevant to future dynamics. In a chess game the full board position is Markov — knowing the board and the next move uniquely determines the distribution over future boards. Only tracking one piece’s position would *not* be Markov, since the hidden pieces still influence legal moves.

A Cleaning Robot as a tiny MDP

The classic toy example. A robot vacuum has two battery states $\mathcal{S} = \{\text{high}, \text{low}\}$. From **high** it can **wait** (sit still, low reward, battery unchanged) or **search** (clean the room, high reward, but might drain to **low**). From **low** it can again **wait** or **search**, but now **search** risks running out of charge entirely — which costs a big penalty (someone has to rescue the robot) and resets it to **high**. From **low** it can also **recharge**: deterministic transition back to **high**, zero reward.

Concretely: $r_{\text{search}} > r_{\text{wait}} > 0$, running out gives reward -3 , and stochastic transition probabilities $\alpha, \beta \in (0, 1)$ control how often searching drains the battery.

Tiny as it is, the example already contains every piece of the MDP framework: a finite state space, a state-dependent action set (**recharge** only available in **low**), stochastic transitions, and a reward function. Once you can write the problem down like this, you can apply any of the algorithms in the rest of this course.

1.3 Goals and Returns

The agent’s objective is to maximize not just the immediate reward but the long-run *return* — the cumulative discounted sum of future rewards.

Discounted Return

The **discounted return** from time t is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1)$ is the **discount factor**. Discounting makes the sum finite for continuing tasks and encodes preference for sooner rewards.

Recursive identity:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Starting from $G_T = 0$ and iterating backwards is far more efficient than summing forward from each t .

The discount factor γ controls the effective time horizon. As $\gamma \rightarrow 0$ only the immediate reward matters (myopic agent); as $\gamma \rightarrow 1$ distant rewards count nearly as much as immediate ones

(far-sighted agent). For **episodic tasks** that terminate at time T the sum is naturally finite and $\gamma = 1$ is permissible. For **continuing tasks** with no terminal time, $\gamma < 1$ is required. A sanity check: for a continuing task with constant reward r and $\gamma < 1$, the geometric series gives $G_t = r/(1 - \gamma)$.

1.4 Policies and Value Functions

The agent's behavior is fully described by its *policy*.

Policy

A **policy** π is a mapping $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ that assigns each state–action pair the probability of selecting a in state s :

$$\pi(a \mid s) = \Pr(A_t = a \mid S_t = s), \quad \sum_{a' \in \mathcal{A}} \pi(a' \mid s) = 1 \quad \forall s \in \mathcal{S}.$$

A policy is *deterministic* if it assigns probability 1 to a single action per state. MDP policies depend only on the current state (Markov sufficiency).

The agent seeks a policy π that maximizes expected return. Two value functions are the central objects for evaluating and improving policies.

State-Value and Action-Value Functions

Given policy π , the **state-value function** and **action-value function** are

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s], \quad q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

$v_\pi(s)$ measures how good it is to *be* in state s under π ; $q_\pi(s, a)$ measures how good it is to *take action* a in s and then follow π . They are related by:

$$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a) = \mathbb{E}_\pi[q_\pi(S_t, A_t) \mid S_t = s].$$

Optimal policies are defined via a partial order on value functions: $\pi \geq \pi'$ iff $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$.

Optimal Value Functions

$$v_*(s) = \max_{\pi} v_\pi(s), \quad q_*(s, a) = \max_{\pi} q_\pi(s, a).$$

The MDP is “solved” when the optimal value function is known.

Optimal Policies

For any finite MDP:

- There exists an optimal policy π^* such that $\pi^* \geq \pi$ for all π .
- All optimal policies achieve v_* and q_* simultaneously.
- There is always a *deterministic* optimal policy.

Given q_* , the optimal policy is recovered by a greedy step:

$$\pi^*(s, a) = \begin{cases} 1 & a = \arg \max_{a'} q_*(s, a'), \\ 0 & \text{otherwise.} \end{cases}$$

If multiple actions share the maximum, any (stochastic) selection among them is optimal. Crucially, knowing q_* yields π^* without needing a model of $p(s', r \mid s, a)$ — a key motivator for action-value methods.

1.5 Exploration vs. Exploitation

A learning agent must balance two competing objectives. **Exploitation**: act on current knowledge to maximize performance. **Exploration**: try actions that increase knowledge, possibly at short-term cost. Neither extreme is optimal: pure exploitation risks committing to a suboptimal action before adequate data are gathered; pure exploration never leverages what is already known.

The tension is cleanest in the *stateless* setting of multi-armed bandits, where no state transitions complicate the picture. Bandits therefore serve as the canonical sandbox for studying exploration strategies before extending them to the full MDP setting.

1.6 Multi-Armed Bandits

1.6.1 Problem setup

k-Armed Bandit

A ***k*-armed bandit** has k actions (“arms”) $a \in \{1, \dots, k\}$. There is a single state (non-associative setting): actions have no long-term consequences on the environment beyond the immediate reward. At each step t the agent selects arm A_t and observes reward R_t . The **true action value** is

$$q(a) = \mathbb{E}[R_t \mid A_t = a].$$

The agent maintains an estimate $Q_t(a)$ and a count $N_t(a)$. True values are unknown and must be learned from experience.

A problem is **stationary** if $q(a)$ is constant over time, and **non-stationary** if the reward distributions drift. This distinction drives the choice of estimation method.

1.6.2 Action value estimation

The natural unbiased estimator of $q(a)$ is the sample average of observed rewards:

$$Q_t(a) = \frac{\sum_{n=1}^t R_n \cdot \mathbb{1}_{A_n=a}}{N_t(a)}.$$

By the law of large numbers, $Q_t(a) \rightarrow q(a)$ as $N_t(a) \rightarrow \infty$. Rather than recomputing from scratch each step, the incremental form processes one observation at a time:

Incremental Update

After selecting A_t and observing R_t , update only the chosen arm's estimate:

$$Q_{t+1}(A_t) = Q_t(A_t) + \alpha_t(R_t - Q_t(A_t)),$$

where $(R_t - Q_t(A_t))$ is the *prediction error*.

- $\alpha_t = 1/N_t(A_t)$: reproduces the unbiased sample average.
- **Constant** $\alpha \in (0, 1]$: exponentially recency-weighted average — older rewards count less, ideal for non-stationary problems.

All other estimates $Q_t(a)$ with $a \neq A_t$ remain unchanged.

1.6.3 Regret

Rather than tracking absolute reward, it is informative to measure how much is *lost* by not always selecting the optimal arm.

Regret

The **optimal value** is $q^* = \max_a q(a)$. The **regret** of selecting action a is

$$\Delta_a = q^* - q(a) \geq 0.$$

The **total regret** after t steps is

$$L_t = \sum_{n=1}^t (q^* - q(A_n)) = \sum_{n=1}^t \Delta_{A_n}.$$

Maximizing cumulative reward is equivalent to minimizing total regret L_t .

A good exploration strategy keeps L_t sub-linear in t — the per-step regret eventually goes to zero. Pure greedy can suffer linear regret if it commits early to a suboptimal arm.

1.6.4 Algorithms

Bandit algorithms split into two families. **Value-based** methods estimate $Q_t(a)$ and derive a policy from those estimates. **Policy-based** methods skip estimation and tune action preferences directly. Below, we walk up the value-based ladder from naive to clever, then look at the policy-based alternative.

Greedy — the cautionary tale.. $A_t \in \arg \max_a Q_t(a)$. Pure exploitation: always pick the arm with the highest current estimate. Trivial to implement, and almost guaranteed to fail. A typical failure mode: the truly best arm gets unlucky on its first pull, drops to the bottom of the leaderboard, and never gets tried again — because greedy never revisits an arm it thinks is bad. The agent locks in on whichever arm happened to look good early. Linear regret.

ϵ -greedy — a constant trickle of exploration.. The simplest fix: most of the time act greedily, but with probability ϵ pick uniformly at random. Now every arm gets sampled forever, so the estimates eventually concentrate on their true values.

ε -Greedy Action Selection

$$A_t = \begin{cases} \arg \max_a Q_t(a), & \text{with probability } 1 - \varepsilon, \\ \text{uniform random from } \{1, \dots, k\}, & \text{with probability } \varepsilon. \end{cases}$$

A fixed $\varepsilon > 0$ guarantees infinite exploration — but also wastes a constant fraction of pulls on random actions forever, even after we have figured out which arm is best. The compromise is a **GLIE** schedule (Greedy in the Limit with Infinite Exploration), like $\varepsilon_k = 1/k$, that decays exploration over time while still sampling each arm infinitely often. Sub-linear regret, asymptotic convergence.

UCB — explore the arms you are unsure about, not the ones you have written off. ε -greedy treats all non-greedy arms equally. That is wasteful: an arm we have pulled 1,000 times and know is bad gets the same exploration probability as one we have pulled twice and barely understand. UCB fixes this by adding an explicit *uncertainty bonus* to each arm:

UCB Action Selection

$$A_t \in \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $c > 0$ controls how aggressively we explore. Pull each arm at least once before applying the formula (the bonus is undefined for $N_t(a) = 0$).

The bonus $c\sqrt{\ln t/N_t(a)}$ shrinks as $N_t(a)$ grows, but never quite to zero — $\ln t$ keeps inching up, so even well-understood arms occasionally get pulled again. The $\sqrt{\ln t/N_t(a)}$ shape is not arbitrary: it comes from a confidence interval on the empirical mean (Hoeffding’s inequality), so the bonus is roughly “a 95%-confidence upper bound on this arm’s true value, given the data so far.” Acting on the upper bound is the principle of **optimism in the face of uncertainty**: when in doubt, act as if the unknown thing might be great — because either it *is* great (and you collect the reward), or it isn’t (and you learn that and stop exploring it).

Why UCB beats ε -greedy

ε -greedy: “with 10% probability, do something random.” UCB: “with 100% probability, do whatever has the highest value-plus-uncertainty.”

The key difference: UCB knows *which* arms it is uncertain about. It will keep sampling an arm that might be best, but it won’t waste pulls on arms it has established are clearly bad. ε -greedy explores blindly; UCB explores strategically.

Thompson Sampling — gambling with posteriors. A Bayesian alternative: maintain a posterior distribution over each $q(a)$ instead of a point estimate. At each step, *sample* one $\tilde{q}(a)$ from each arm’s posterior, then act greedily on the samples.

- Arms with high posterior variance (we are uncertain about them) produce widely varying samples, sometimes large enough to win the argmax. Those arms get pulled.
- Arms whose posterior is sharply peaked around a low value almost never produce a high sample, so they are ignored.

UCB vs. Thompson Sampling

Both methods are uncertainty-aware, just with different decision rules. **UCB** is deterministic and optimistic: “I’ll pick the arm whose *best plausible* value is highest.” **Thompson** is probabilistic and Bayesian: “I’ll pretend one specific value function is true (sampled from my beliefs) and act greedily on that.” Both achieve near-optimal regret. Thompson tends to outperform UCB on real problems because its randomization is naturally calibrated to the posterior, while UCB’s $c\sqrt{\ln t/N_t}$ bonus is just a worst-case bound. This is the same UCB-vs.-Thompson choice we will see again in Chapter 6 when discussing deep-RL exploration.

Gradient Bandits (policy-based).

Preference Parameters

Instead of estimating action values, gradient bandits maintain a real-valued **preference** $H_t(a) \in \mathbb{R}$ for each arm. Only *differences* matter: adding a constant to all preferences leaves the policy unchanged.

Preferences are converted to a sampling distribution via the softmax:

$$\pi_t(a) = \frac{\exp(H_t(a))}{\sum_b \exp(H_t(b))}.$$

After observing reward R_t with running baseline $\bar{R}_t$ (often the running average reward), preferences are updated by stochastic gradient ascent on expected reward:

$$H_{t+1}(a) = \begin{cases} H_t(a) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(a)) & a = A_t, \\ H_t(a) - \alpha(R_t - \bar{R}_t)\pi_t(a) & a \neq A_t. \end{cases}$$

What the Gradient Update Does

Think of π_t as a weighted lottery. If $R_t > \bar{R}_t$ (better than average): increase $H(A_t)$, making the selected arm more likely; slightly decrease the rest. If $R_t < \bar{R}_t$: decrease $H(A_t)$ and increase the rest. The factors $(1 - \pi_t(A_t))$ and $\pi_t(a)$ scale the update: a surprising action (low π_t) triggers a large adjustment; a near-certain action changes little. Unlike value-based methods, gradient bandits have no explicit exploration rate — exploration emerges from the softmax sampling itself.

Tabular Value-Based Methods

Lecture 02 is organized around a single central question: *how do we actually solve for an optimal policy?* The answer depends on two things — whether we have access to a model of the environment, and whether we compute expectations exactly or estimate them from samples. Those two choices carve out the three algorithm families covered here: Dynamic Programming, Monte Carlo, and Temporal-Difference learning.

2.1 RL Taxonomy

Before diving into algorithms, it helps to have a mental map of where each one sits. The RL landscape can be carved up along three roughly independent axes: *what* the agent learns, *whose* data it learns from, and *whether it has a model* of the environment.

What gets learned.. A method is **value-based** if it learns a value function and reads the policy off it (“in this state, the action with the highest Q-value is best, so do that”). **Policy-based** methods skip the value function and directly tune a policy; they answer “what should I do?” without ever asking “how good is this state?” **Actor-critic** methods do both — they keep a policy (the actor that picks actions) and a value function (the critic that judges them), and use the critic’s opinions to improve the actor.

Whose data.. **On-policy** methods are puritans: the policy they evaluate has to be the policy that collected the data. As soon as you change the policy, the old data is officially stale. **Off-policy** methods are pragmatists: the *target* policy π (what we want to learn) and the *behavior* policy μ (what actually collected the data) can be different. That gap is what lets us reuse old experience, learn from human demonstrations, or learn a greedy policy while still exploring. **Offline** methods go to the extreme: a fixed dataset, no environment interaction at all.

Model or no model.. **Model-based** methods use a model of the dynamics $p(s', r \mid s, a)$ — either given to them or learned — and can plan by simulating transitions. **Model-free** methods just learn from interaction, never bothering to predict what the environment will do next.

Composing the axes

The three axes are largely independent, so any combination is possible:

- **DQN**: value-based, off-policy, model-free.
- **REINFORCE**: policy-based, on-policy, model-free.
- **PPO / SAC**: actor-critic, on/off-policy, model-free.

- **AlphaZero**: actor-critic, on-policy, model-based (it has a perfect chess simulator and uses it to plan).

This chapter lives in the value-based + model-free corner of the cube.

2.2 Bellman Equations

The Bellman equations are the mathematical backbone of tabular RL. They express a simple recursive insight: the value of being in a state equals the immediate reward you expect plus the discounted value of where you end up. This sounds almost trivial, but it is the engine that drives almost every algorithm in this chapter.

Where the recursion comes from

Take the definition of the return, $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$, and pull out the first reward:

$$G_t = R_{t+1} + \gamma \underbrace{(R_{t+2} + \gamma R_{t+3} + \dots)}_{=G_{t+1}}.$$

Take expectations on both sides given $S_t = s$ and a policy π , and the value function “unrolls itself by one step”:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s].$$

That is the entire content of the Bellman expectation equation. Everything below is just spelling out the expectation as a sum over actions and next states.

Bellman Expectation and Optimality Equations

For any policy π , the value functions satisfy the *expectation equations*:

$$v_\pi(s) = \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v_\pi(s') \right],$$

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_\pi(s', a').$$

The *optimal* value functions satisfy the *optimality equations*:

$$v_*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) v_*(s') \right],$$

$$q_*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \max_{a'} q_*(s', a').$$

The expectation equations are *linear* in the value function — in principle you could solve them as a linear system in $O(|\mathcal{S}|^3)$ operations. For anything but tiny MDPs that is computationally prohibitive, so iterative methods take over.

The optimality equations look almost the same but replace the policy-weighted average with a max. That max makes them nonlinear, so there is no closed-form solution — but it also gives them a beautiful interpretation: the optimal value of a state is what you get if you act greedily forever after this step.

Why q_* is more useful than v_*

Once you have q_* , the optimal policy is free: $\pi^*(s) = \arg \max_a q_*(s, a)$. Just look up which action has the highest action-value and take it.

If you only have v_* , you have to compute $\arg \max_a [r(s, a) + \gamma \sum_{s'} p(s' | s, a) v_*(s')] —$ which requires the environment model. This is why action-value methods are so attractive in model-free settings: q_* tells you what to do without consulting any dynamics.

2.3 Planning with Dynamic Programming

Dynamic programming computes optimal policies *given a perfect model* of the environment. DP uses the model to form exact expected-value backups rather than sampling. Every DP algorithm alternates between two steps: **policy evaluation** (compute how good the current policy is) and **policy improvement** (make the policy better).

2.3.1 Policy evaluation and improvement

Given a fixed policy π , we want to compute its value function v_π exactly. The trick is to treat the Bellman expectation equation not as a system of equations to solve, but as an *update rule*. Starting from any initialization V_0 , iterating the update

$$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \right]$$

for all states s simultaneously converges to v_π . In practice we stop when the maximum change across states falls below a small threshold ϵ .

Once we have $V \approx v_\pi$, policy improvement is straightforward. The **policy improvement theorem** guarantees that choosing the greedy action at every state — $\pi'(s) = \arg \max_a q_\pi(s, a)$ — always yields a policy at least as good as π . If $v_{\pi'}(s) = v_\pi(s)$ for all s , the policy was already optimal.

2.3.2 Policy iteration and value iteration

Policy iteration repeats evaluation and improvement in alternation: run iterative policy evaluation to convergence, extract the greedy policy, repeat. It is guaranteed to reach the optimal policy after a finite number of improvements, but running evaluation fully to convergence on each iteration is expensive.

Value iteration is a smarter shortcut: do not wait for evaluation to converge. Instead, apply a *single* backup using the optimality operator — taking the max rather than the policy-weighted sum:

$$V_{k+1}(s) \leftarrow \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' | s, a) V_k(s') \right].$$

This is equivalent to interleaving one step of evaluation with one step of improvement: every iteration both evaluates and improves. Value iteration converges asymptotically (no finite termination guarantee) but is typically more efficient per iteration than running full policy evaluation.

Value iteration unrolling, by hand

Imagine a tiny three-state corridor $s_1 - s_2 - s_3$, with s_3 a terminal goal giving reward +1 and s_1, s_2 giving reward 0. Actions move deterministically left or right. Discount $\gamma = 0.9$. Initialize $V_0 = (0, 0, 0)$.

Iteration 1. For each non-terminal state, take the best action.

- $V_1(s_2) = \max\{0 + 0.9 \cdot V_0(s_1), 1 + 0.9 \cdot V_0(s_3)\} = 1$ (go right).
- $V_1(s_1) = \max\{0 + 0.9 \cdot V_0(s_2), \dots\} = 0$.

Iteration 2.

- $V_2(s_2) = 1 + 0.9 \cdot 0 = 1$ (still optimal to step right and collect).
- $V_2(s_1) = 0 + 0.9 \cdot 1 = 0.9$ (one step toward s_2 , then collect).

The reward propagates one step backward per iteration — after k iterations every state within k steps of the goal has correct value. The greedy policy “always move toward s_3 ” is recovered immediately.

That is the entire algorithm: forward Bellman backups, reward seeping backward through the state graph one step at a time, until the value function stops changing.

Limitation of DP

Both algorithms require knowing $p(s', r \mid s, a)$ for every transition. In most real-world settings this model is unavailable. Model-free methods, covered next, replace the model-based expectation with samples drawn from the environment — giving up exactness for the ability to learn from raw experience.

2.4 Model-Free Prediction

The prediction task is: given a policy π , estimate v_π from experience alone, without any model. Two fundamentally different approaches exist.

2.4.1 Monte Carlo: learn from complete episodes

Monte Carlo (MC) methods estimate $v_\pi(s)$ by averaging the actual returns observed when starting from s . Since $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and G_t is observable at episode end, we can use the sample average as the estimator, updated incrementally:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)].$$

G_t is an *unbiased* estimate of $v_\pi(S_t)$ — on average it equals the true value. The downside is variance: G_t is a sum of many random rewards, so individual estimates can vary wildly, and the method requires waiting until the episode ends.

2.4.2 Temporal-difference: learn online, one step at a time

TD methods trade unbiasedness for immediacy. Instead of waiting for the actual return G_t , TD uses the Bellman equation to construct an estimate after just one step — the **TD(0) target** $R_{t+1} + \gamma V(S_{t+1})$. The update is:

TD(0) Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD error } \delta_t}.$$

TD(0) is model-free like MC, bootstraps like DP, and can update after *every step* without waiting for episode end.

The TD target is *biased* because $V(S_{t+1})$ is only an approximation — we are plugging an estimate back into its own update. But it has far lower variance than G_t , because only a single reward is random rather than a whole trajectory’s worth. In practice, TD often learns faster even though it introduces this bias.

Information propagation differs too. In MC, a surprising outcome at the end of an episode immediately updates all states visited along the way. In TD, that information trickles backward only one step at a time.

2.4.3 Multi-step returns and TD(λ)

n -step TD provides a smooth interpolation between TD(0) ($n = 1$) and MC ($n = \infty$) by collecting n actual rewards before bootstrapping:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Rather than committing to one n , TD(λ) mixes *all* n -step returns simultaneously using geometrically decaying weights:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_t^{(n)} + \lambda^{T-t-1} G_t.$$

Setting $\lambda = 0$ recovers TD(0); $\lambda = 1$ recovers MC. Intermediate values get the best of both worlds: lower variance than MC while propagating information faster than one-step TD. In practice the λ -return is implemented online via **eligibility traces**, which maintain a decaying memory of recently visited states and spread the TD error backward proportionally.

2.5 Model-Free Control

For control, we need action-value estimates $Q(s, a)$ rather than state values. The reason is simple: to be greedy over v_π , you need to compute $\arg \max_a [r(s, a) + \gamma \sum_{s'} p(s' | s, a) V(s')]$, which requires a model. Being greedy over $Q(s, a)$ is model-free: just pick $\arg \max_a Q(s, a)$.

The general framework is **generalized policy iteration**: alternate between approximate policy evaluation (using MC or TD to estimate $Q \approx q_\pi$) and policy improvement (act greedily with respect to Q). The catch is exploration: a purely greedy policy never tries suboptimal actions, so $Q(s, a)$ for those actions never gets updated and the policy can get stuck. The standard fix is ε -greedy: with probability ε take a random action, otherwise act greedily. To guarantee convergence to the optimal policy, we need the exploration schedule to satisfy GLIE — *Greedy in the Limit with Infinite Exploration*: every state-action pair must be visited infinitely often, yet the policy must eventually converge to greedy. An ε -greedy schedule with $\varepsilon_t \rightarrow 0$ and $\sum_t \varepsilon_t = \infty$ satisfies GLIE.

2.5.1 SARSA: on-policy TD control

SARSA applies the TD idea directly to action values. At each step we observe the transition $(S_t, A_t, R_{t+1}, S_{t+1})$, then sample the next action A_{t+1} from the current policy and update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

The five quantities $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ give the algorithm its name. SARSA is *on-policy*: the action A_{t+1} used in the target is actually sampled from the current ε -greedy policy, so the estimate converges to q_π for the policy being followed.

A variant, **Expected SARSA**, replaces the sampled A_{t+1} with an expectation over the policy:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

This reduces variance because it averages over the randomness in A_{t+1} rather than sampling it.

2.5.2 Q-learning: off-policy TD control

Q-learning breaks the link between behavior and learning target. Regardless of which action the agent *actually* takes next, the update uses the *greedy* action at the next state:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right].$$

This is a sample-based approximation of the Bellman optimality equation for q_* , and under standard conditions Q converges to q_* regardless of the behavior policy — the behavior just needs to ensure every state-action pair gets visited. No importance-sampling correction is needed because the max target does not depend on the behavior policy at all.

In one sentence: SARSA evaluates what the agent *actually does*; Q-learning evaluates what the agent *would do if it were greedy*.

Cliff Walking: SARSA vs. Q-learning

A classic gridworld where the bottom row is a cliff: stepping into it gives a big negative reward and resets the agent to the start. The shortest path runs along the edge of the cliff; a longer, safer path goes through the top of the grid.

Train both algorithms with ε -greedy exploration ($\varepsilon = 0.1$).

- **Q-learning** learns the *optimal* (cliff-edge) policy: its target uses $\max_{a'}$, ignoring the fact that during exploration the agent will occasionally take a random action — and falling off the cliff while exploring along the edge hurts. Q-learning's Q -values reflect the value of *playing perfectly*, even when the actual behavior is noisy.
- **SARSA** learns the *safe* policy: its target uses the next action A_{t+1} that the policy actually picks — which, with ε chance, is random. Walking the cliff edge with random actions costs a lot in expectation, so SARSA's Q -values penalize it and the policy detours through the safer top row.

The lesson: SARSA optimizes the policy you're currently using (including its noise); Q-learning optimizes the greedy policy you'd play *if* you stopped exploring. Once ε is annealed to zero, both converge to the same optimal solution — but their behavior during training is different in a way that actually matters in risky environments.

2.5.3 Maximization bias and Double Q-learning

There is a subtle problem with the max in Q-learning. When Q -values are estimated with noise — as they always are from finite data — taking the maximum over noisy estimates produces a value that is systematically *too high*. The noise pushes the max upward. This is called **maximization bias**.

Why max gets it wrong (concrete numbers)

Suppose three actions in some state really have value 0, but each Q -estimate is true value plus noise $\mathcal{N}(0, 1)$. So on any given update:

- $Q(s, a_1), Q(s, a_2), Q(s, a_3)$ are all centered at zero on average.
- $\mathbb{E}[\max_a Q(s, a)] \approx 0.85 > 0$ — the maximum of three independent standard Gaussians is biased high.

Q-learning's TD target uses this maximum, so it inflates value estimates by roughly 0.85. The error compounds: inflated values get bootstrapped into neighboring states, which inflates them too, which inflates the original state more on the next update, and so on. The bias is worst exactly when it hurts most — early in training, when noise is high and the agent commits to whichever action got lucky on the first few samples.

The fix is to *decouple action selection from action evaluation*, so the noise in selection doesn't propagate into the evaluated value.

Double Q-Learning

Maintain two independent estimates Q^A and Q^B . Use one to *select* the action and the other to *evaluate* it:

$$Q^A(S_t, A_t) \leftarrow Q^A(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q^B(S_{t+1}, \arg \max_a Q^A(S_{t+1}, a)) - Q^A(S_t, A_t) \right].$$

At each step, flip a coin and update either Q^A or Q^B ; the other plays the evaluation role. Decoupling selection from evaluation breaks the positive correlation that drives the bias upward: Q^A might overestimate a_1 , but Q^B 's independent estimate of a_1 is just as likely to be too low.

2.5.4 Off-policy learning and importance sampling

In off-policy learning, the agent collects data with a *behavior policy* μ (maybe ε -greedy, or human demonstrations, or a previous version of itself) but wants to learn about a different *target policy* π (maybe the greedy one). The catch is that any expectation we compute from μ -collected data is, by default, an expectation under μ — not the π we care about.

Importance sampling corrects this distribution mismatch by reweighting each sample according to how likely it would be under the target policy:

$$\rho_t = \frac{\pi(A_t | S_t)}{\mu(A_t | S_t)}.$$

A transition where π would have taken this action much more often than μ gets a large weight; one where π would rarely have chosen this action gets a small weight.

Reading the importance ratio

$\rho_t > 1$: “the target would have done this more often than the behavior — count this transition for more.”

$\rho_t < 1$: “the target would have done this less often — count it for less.”

$\rho_t = 0$: “the target would never have done this — ignore the transition.”

Different methods need different amounts of correction. **Off-policy MC** reweights the entire return by the product of ratios over the whole trajectory, $\rho_t \rho_{t+1} \cdots \rho_{T-1}$ — one mismatch anywhere in the future contaminates the credit at t . **Off-policy SARSA** only needs a single-step ratio ρ_{t+1} , because the TD target only looks one step ahead. **Q-learning** doesn’t need any importance correction: its $\max_{a'}$ target ignores which action the behavior policy actually took, so there is no mismatch to correct. This is one reason Q-learning is the standard off-policy method — it avoids the high-variance ratio products that haunt MC-style off-policy learning. We will see the price of those products again in Chapter 5 when we look at off-policy policy gradients.

Value Function Approximation

Every tabular method we have seen so far stores a separate number for each state or state-action pair. That is fine for toy problems, but it breaks at scale. Backgammon has around 10^{20} states; Go has around 10^{170} ; robotic manipulation lives in a continuous space with infinitely many states. No table can hold all of those, and no algorithm could visit each state enough times to learn a reliable estimate.

The solution is to give up on exact representation and *approximate* the value function with a parameterized function $v_{\mathbf{w}}(s) \approx v_{\pi}(s)$, where $\mathbf{w}$ is a weight vector with far fewer entries than there are states. The approximation generalizes: updating $\mathbf{w}$ from one state implicitly updates the estimated values of nearby states that share features. When neural networks play the role of this approximator, the whole framework becomes **deep reinforcement learning**.

3.1 Learning Value Functions by Gradient Descent

The natural objective is to minimize the mean-squared difference between the approximate and the true value function over states visited under π :

$$J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - v_{\mathbf{w}}(S))^2 \right].$$

Stochastic gradient descent gives the parameter update at each visited state S :

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S) - v_{\mathbf{w}}(S)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S).$$

The problem is that $v_{\pi}(S)$ is unknown — we can never query the oracle. So we substitute it with a **target** derived from observed data. The MC target is the actual return G_t (unbiased but noisy). The TD(0) target is the one-step bootstrapped estimate $R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$ (lower variance, but biased because it depends on $\mathbf{w}$ itself). The TD(λ) target G_t^{λ} sits between the two extremes.

This dependency is important: the TD target changes as $\mathbf{w}$ updates, so we are not performing true gradient descent — we are ignoring the gradient through the target. This is called a **semi-gradient** method. It works well empirically, but convergence guarantees are weaker than for full gradient descent, and there are regimes where it can diverge (see the Deadly Triad, below).

3.2 Function Approximators

Linear approximation represents each state by a feature vector $\mathbf{x}(s) \in \mathbb{R}^d$ and computes the value as a dot product: $v_{\mathbf{w}}(s) = \mathbf{x}(s)^{\top} \mathbf{w}$. The squared-error objective is then quadratic in $\mathbf{w}$,

guaranteeing a unique global minimum, and the SGD update simplifies to $\Delta \mathbf{w} = \alpha(\text{target} - v_{\mathbf{w}}(S))\mathbf{x}(S)$. Tabular lookup is actually a special case of linear approximation: represent each state with a one-hot vector, and the weights $\mathbf{w}$ directly encode the value table. The difference is that one-hot features provide no generalization between states, whereas a shared feature representation does.

Challenges Specific to RL

Unlike supervised learning, function approximation in RL faces three compounding difficulties: **non-i.i.d. data** (consecutive transitions are strongly correlated, not independent samples); **policy-dependent data distribution** (the states the agent visits change as the policy improves); and **non-stationary targets** (in TD methods, the regression target changes every time $\mathbf{w}$ is updated, because the target network depends on the same weights being trained). Together these can destabilize learning, especially with nonlinear approximators.

In practice, the choice of approximator involves a three-way trade-off. *Tabular* methods have the best theory but cannot scale. *Linear* methods have reasonably strong theory but require hand-crafted features. *Nonlinear* methods — especially deep neural networks — lack tight theoretical guarantees but scale well and eliminate the need to design features by hand. Neural networks are the dominant choice in modern deep RL.

3.3 Action-Value Approximation and Control

For control we approximate $q_{\pi}(s, a)$ rather than $v_{\pi}(s)$, because the greedy improvement step $\pi'(s) = \arg \max_a q_{\mathbf{w}}(s, a)$ then requires no model. There are two natural network architectures. *Action-in* takes (s, a) as input and produces a single Q-value — you need one forward pass per action to find the best one. *Action-out* takes s as input and outputs a Q-value for every action simultaneously — much more efficient when the action space is discrete.

The same MC / TD(0) / TD(λ) targets used for state values apply directly to action values: the oracle $q_{\pi}(S, A)$ is replaced by G_t , by $R_{t+1} + \gamma q_{\mathbf{w}}(S_{t+1}, A_{t+1})$, or by G_t^{λ} . The control loop then mirrors GPI: approximate policy evaluation followed by ε -greedy improvement.

The Deadly Triad

Combining **function approximation**, **bootstrapping** (TD-style targets), and **off-policy learning** in the same algorithm can cause divergence. Each ingredient alone is fine; together they can form a feedback loop where weights drift to infinity. MC avoids it by not bootstrapping; on-policy TD with linear approximation avoids it by restricting the data distribution. Non-linear, off-policy TD — the core of DQN-style algorithms — sits right inside the danger zone, which is why DQN needs special stabilization tricks.

3.4 Batch Learning and Experience Replay

Plain online SGD is sample-inefficient: each transition is seen once and then discarded. Worse, consecutive transitions are highly correlated, which makes the gradient estimates noisy and can cause oscillations.

Experience replay addresses both problems. The agent stores all transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ in a **replay buffer** $\mathcal{D}$. At each training step, a random mini-batch is sampled from $\mathcal{D}$ and used

to compute a gradient update:

$$\Delta \mathbf{w} = \alpha (r + \gamma v_{\mathbf{w}}(s') - v_{\mathbf{w}}(s)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(s).$$

Random sampling breaks the temporal correlations. Each transition can be replayed many times, improving sample efficiency. The buffer also acts as a stabilizing data distribution: because it contains transitions from many past policies, the training distribution changes slowly even as the current policy evolves quickly.

3.5 Deep Q-Networks

Applying Q-learning directly with a neural network runs into trouble. Consecutive transitions are correlated (SGD assumes i.i.d. data). The TD target $r + \gamma \max_{a'} Q_{\mathbf{w}}(s', a')$ depends on $\mathbf{w}$, so every weight update shifts the target — like trying to hit a moving goalpost. And the combination of off-policy data, function approximation, and bootstrapping puts us squarely in the deadly triad.

DQN (Mnih et al., 2015) tames all three with two techniques. **Experience replay** (described above) breaks correlations and enables data reuse. **Fixed target networks** address the moving-goalpost problem: maintain a second, frozen network $Q_{\mathbf{w}^-}$ and compute targets from it, $y = r + \gamma \max_{a'} Q_{\mathbf{w}^-}(s', a')$, only periodically copying $\mathbf{w}$ into $\mathbf{w}^-$. Between copies the targets are stable, making training resemble supervised learning.

DQN Training Loop

For each step: collect $(s_t, a_t, r_{t+1}, s_{t+1})$ with ε -greedy and store in $\mathcal{D}$. Sample a mini-batch $\{(s_j, a_j, r_j, s'_j)\}_{j=1}^B$. Compute targets $y_j = r_j + \gamma \max_{a'} Q_{\mathbf{w}^-}(s'_j, a')$ (or $y_j = r_j$ if terminal). Minimize $L(\mathbf{w}) = \frac{1}{B} \sum_j (y_j - Q_{\mathbf{w}}(s_j, a_j))^2$ w.r.t. $\mathbf{w}$ only (target y_j treated as a constant). Every C steps: $\mathbf{w}^- \leftarrow \tau \mathbf{w} + (1 - \tau) \mathbf{w}^-$.

DQN was evaluated on 49 Atari 2600 games with *the same architecture and hyperparameters* across all games. Raw pixels were preprocessed to 84×84 grayscale; four frames were stacked to give the network a sense of motion; a CNN processed these into Q-values for each action. DQN outperformed all prior RL methods on 43 of 49 games and reached human-level performance on 29 of them — the first demonstration of general-purpose deep RL at scale.

3.6 Beyond DQN: Key Extensions

DQN was a breakthrough but not the end of the story — six targeted improvements plug six different weaknesses, and **Rainbow** (Hessel et al., 2018) combines all of them into one agent that reaches DQN's final performance in roughly $7 \times$ fewer environment steps. The interesting thing about Rainbow is that no single ingredient is responsible for the gain — ablating any one of them noticeably hurts. Each extension fixes a different bottleneck.

Double DQN — fixing maximization bias. DQN's target $y = r + \gamma \max_{a'} Q_{\mathbf{w}^-}(s', a')$ uses the *same* frozen network to pick the best action and to evaluate its value. Whichever action got an inflated estimate from noise gets selected, and that inflated estimate is exactly the value used in the target. Same maximization bias as tabular Q-learning, just with a neural network making the noise. Double DQN decouples the two roles: the *online* network selects the action, $a^* = \arg \max_{a'} Q_{\mathbf{w}}(s', a')$, and the *target* network evaluates it, $y = r + \gamma Q_{\mathbf{w}^-}(s', a^*)$. The two networks have correlated noise but not identical noise, so the worst of the upward bias is removed — and DQN's notorious early-training overestimation largely disappears.

Prioritized experience replay — mining the buffer for surprises.. Uniform replay treats every transition the same. But some transitions are *boring* — the network already predicts them perfectly — and some are *informative* — the network still gets them wrong. Sampling everything uniformly wastes most of the gradient budget on transitions the network already understands. Prioritized replay samples each transition i with probability proportional to $|\delta_i|^\alpha / \sum_k |\delta_k|^\alpha$, where δ_i is the TD error (“how surprised was the network?”). Surprises get replayed more often. Because this biases the data distribution, an importance-sampling weight $w_i \propto (1/p_i)^\beta$ corrects the resulting bias in the gradient. Result: faster learning, especially in sparse-reward Atari games where successful trajectories are rare.

Dueling networks — separating “where am I” from “what should I do”.. A Q-value bundles two questions into one number: how good is this state, and how much better is this action than typical? Sometimes the first question dominates (you’re winning, every reasonable action keeps the lead) and sometimes the second does (the position is neutral, but one specific move wins). Mixing both into a single Q-head means the network spends capacity learning the same “state value” redundantly across all actions. A dueling architecture has a shared backbone that splits into two heads — one for the state value $V(s)$ and one for the per-action advantage $A(s, a)$ — and recombines them as $Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a')$. The mean-subtraction term enforces an identifiability constraint and avoids the two streams drifting in lockstep. The big practical win: the value head can still be trained from *every* transition even when a particular action wasn’t taken — so generalization across states improves, especially when many actions have similar value.

Multi-step returns — propagating reward faster.. 1-step TD ($n = 1$) is conservative: a reward at time t takes one update to reach $t - 1$, two to reach $t - 2$, and so on. In sparse-reward games (where reward might be hundreds of steps away from the action that caused it) this is glacial. n -step returns $y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \max_{a'} Q_{\mathbf{w}^-}(S_{t+n}, a')$ collect n actual rewards before bootstrapping, propagating credit n steps in one go. The trade-off is variance (more random rewards summed in) and the fact that the target becomes slightly off-policy (the n -step rollout was generated by the old policy). Intermediate values like $n = 3$ to 5 are usually a sweet spot.

Distributional RL — modeling the whole distribution, not just the mean.. Standard Q-learning learns $\mathbb{E}[G_t | s, a]$ — a single scalar. But two actions can have the same expected return while one is consistently mediocre and the other is a high-variance gamble between disaster and triumph. The mean throws that away. Distributional RL learns the entire *distribution* of returns $Z(s, a)$, satisfying the distributional Bellman equation $Z(s, a) \stackrel{d}{=} R + \gamma Z(s', a')$. The agent ends up modeling outcomes as probability distributions over a set of “atoms” (e.g. 51 discrete return levels, hence the original method’s name C51). Empirically, training with a cross-entropy loss on these distributions produces much richer representations — and improves performance even though we still extract a deterministic policy by taking the mean’s argmax.

Noisy Networks — learning to explore.. ε -greedy is uniform: the same noise level everywhere, regardless of whether the agent is confident or confused. That is a bad use of exploration budget. Noisy Networks replace deterministic linear layers with stochastic ones:

$$y = (\mu^w + \sigma^w \odot \varepsilon^w) x + (\mu^b + \sigma^b \odot \varepsilon^b), \quad \varepsilon \sim \mathcal{N}(0, I).$$

The noise scales σ^w, σ^b are *learned* alongside the means. Where the network is confident it shrinks σ toward zero (less exploration); where it is uncertain it keeps σ large (more exploration). This produces *state-dependent* exploration without a hand-tuned ε schedule — and on hard-exploration Atari games it makes a real difference.

Why six fixes, not one?

Each Rainbow ingredient targets a separate failure mode of vanilla DQN: maximization bias (Double DQN), inefficient sampling (prioritized replay), wasted capacity in the value head (dueling), slow credit propagation (multi-step), information loss in the mean (distributional), and dumb exploration (noisy nets). None of these is a magic bullet by itself. Together they cover most of the obvious ways DQN leaks performance, which is why Rainbow's gain is greater than the sum of the individual contributions.

Policy-Based Methods

So far every method in this course has learned a value function and then derived a policy from it — usually ε -greedy w.r.t. a learned $q(s, a)$. This chapter takes the opposite route: we *directly parameterize the policy itself* and learn its parameters by gradient ascent on the expected return. A value function may still appear (as a baseline, or as a critic) but it is no longer the primary object of learning.

The motivation for this shift is concrete: there are tasks where value-based RL is a poor fit, and policy-based methods handle them naturally.

4.1 Why Bother With Policies Directly?

Recall the three families of RL methods:

- **Value-based:** learn $q(s, a)$, derive $\pi(s) = \arg \max_a q(s, a)$.
- **Policy-based:** learn $\pi_\theta(a | s)$ directly — no value function.
- **Actor-critic:** learn both — a parameterized policy (the actor) and a value function (the critic) that helps train the actor.

Value-based methods are great when actions are discrete and the optimal policy is essentially deterministic. They run into trouble in two specific situations.

4.1.1 Continuous action spaces

Many real control problems have *continuous* actions: a robot arm's joint torques live in $\mathbb{R}^d$, a car's steering and throttle are real numbers, locomotion involves dozens of simultaneous motor commands. Deriving a greedy policy in these settings means solving

$$a^*(s) = \arg \max_{a \in \mathbb{R}^d} q(s, a)$$

at every step. For a handful of discrete actions this is trivial — compute $q(s, a)$ for each one and pick the largest. In a continuous space it is a global optimization problem inside the inner loop of training. There is no easy way to do it.

A parameterized policy sidesteps the issue entirely: instead of finding the argmax of a learned function, we just *output* the action (or a distribution over actions) directly from a network.

4.1.2 Deterministic policies are not always enough

Value-based control with greedy improvement converges to a *deterministic* policy — one action per state. For most problems, that is fine. For some, it is fatal.

Rock–Paper–Scissors

The optimal policy in rock-paper-scissors against a savvy opponent is to play each move with probability $\frac{1}{3}$ — the uniform Nash equilibrium. Any deterministic policy is trivially exploitable: play rock every time and your opponent will eventually figure out to play paper. Value-based RL cannot represent the optimal policy here, because greedy improvement always collapses to a single action.

Aliased Gridworld

A 1D gridworld where the agent’s state representation is the binary feature vector

$$\phi(s) = [\text{wall up}, \text{wall right}, \text{wall down}, \text{wall left}].$$

The treasure sits in the middle, between two skull squares. The two “corridor” states on either side of the treasure look identical to the agent: same walls in the same directions. They are *aliased*.

Any deterministic policy on this aliased feature must take the same action in both corridor states. If it picks “move left,” the agent gets stuck on the left side; “move right” and it gets stuck on the right. Either way the agent never reliably finds the treasure.

A *stochastic* policy that flips a coin in those aliased states ($\pi(\text{left}) = \pi(\text{right}) = 0.5$) reaches the treasure quickly with high probability, because the random walk is short and bounded. Direct policy learning can represent exactly this kind of policy.

4.1.3 Interlude: POMDPs

The aliased gridworld is a small instance of a much bigger phenomenon. Up to now we have assumed the agent observes the true state S_t — a **fully observable MDP**. In practice the agent often sees only a partial **observation** $O_t \sim p(o | S_t)$, with the actual state S_t hidden. This is a **Partially Observable MDP (POMDP)**.

Once observations no longer contain the full state, the optimal action may depend on history (memory) or randomization (a stochastic policy). This course mostly ignores memory — but allowing stochastic policies is essentially free with policy-based methods, and it solves a real subset of POMDP problems.

Intuition & Mental Model

The two reasons to want a stochastic policy are subtly different:

- **Game-theoretic:** the optimal policy is genuinely mixed (rock-paper-scissors).
- **Aliasing:** our state representation is incomplete, so randomization is a cheap proxy for the missing information.

Both are out of reach for value-based control. Both are easy for policy-based methods.

4.1.4 Trade-offs

Policy-based methods are not strictly better than value-based ones — they buy flexibility at a cost.

Advantages.. They optimize the true objective (expected return) directly, with no value-function intermediary. They handle continuous actions naturally. They can represent stochastic policies. Sometimes the policy is much simpler than the value function (think “always go left” versus the maze of values that justifies it), and learning the simpler thing is easier. The choice of policy class can also encode prior knowledge about the problem’s structure.

Disadvantages.. They typically have higher *variance* than value-based methods — which we’ll see directly when we look at the policy gradient. They can be less sample-efficient, get stuck in local optima, and the knowledge they learn tends to be specific to the policy class chosen — it generalizes less than a value function would.

4.2 Setting Up the Optimization

A parameterized policy $\pi_\theta(a \mid s)$ is just a function from states to action distributions, with parameters θ (e.g. the weights of a neural network). The job is to find the parameters that maximize policy quality — but first we need to define what “quality” means. There are two natural choices, depending on whether the task is episodic or continuing.

4.2.1 Episodic objective: average return per episode

For episodic tasks we measure the policy by the expected return from the starting-state distribution d_0 :

$$J(\theta) = \mathbb{E}_{S_0 \sim d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t R_t \right] = \mathbb{E}_{S_0 \sim d_0} [v_{\pi_\theta}(S_0)].$$

In words: *the value of the start state, averaged over where the start state can be.*

4.2.2 Continuing objective: average reward per step

For continuing tasks the discounted episodic objective doesn’t quite work — the sum runs to infinity and the boundary effects of S_0 vanish. We use the average reward per step instead:

$$J(\theta) = \mathbb{E}_{\pi_\theta} [R_{t+1}] = \sum_s d_{\pi_\theta}(s) \sum_a \pi_\theta(a \mid s) \sum_r p(r \mid s, a) r,$$

where $d_{\pi_\theta}(s)$ is the *stationary distribution* over states under π_θ — the long-run fraction of time the agent spends in s . This is the expected reward in the states the policy visits.

For the rest of this chapter, treat both objectives as “the expected return $J(\theta)$.” The mathematical machinery that follows applies to both.

4.2.3 Optimization choices

Once we have $J(\theta)$, finding $\theta^* = \arg \max_\theta J(\theta)$ is just optimization. There are two broad classes of methods.

- **Gradient-free:** hill climbing, simplex / Nelder-Mead, genetic algorithms. Useful when the policy isn't differentiable, but not competitive in the high-dimensional regime where neural-network policies live.
- **Gradient-based:** gradient ascent, conjugate gradient, quasi-Newton. These require a way to compute or estimate $\nabla_{\theta} J(\theta)$ — and that is where things get interesting.

The rest of the chapter is about the gradient route.

4.3 Computing the Policy Gradient

The vanilla update is

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta),$$

identical to ordinary gradient ascent. The challenge is that $J(\theta)$ is an *expectation over trajectories*, and the trajectories themselves depend on θ . We can't write down a closed form — we need a way to estimate the gradient from samples.

Three approaches:

- **Finite differences:** perturb each parameter individually and look at the change in J .
- **Score function / log-likelihood trick:** a clever algebraic identity that converts a gradient of an expectation into an expectation of a gradient.
- **Policy gradient theorem:** the score-function trick, applied to the full multi-step MDP setting.

4.3.1 Finite differences

The crudest approach is to estimate each partial derivative numerically by perturbing the parameter:

$$\frac{\partial J(\theta)}{\partial \theta_i} \approx \frac{J(\theta + \varepsilon e_i) - J(\theta - \varepsilon e_i)}{2\varepsilon},$$

where e_i is the unit vector in the i -th direction.

This is simple and works for any policy — even non-differentiable ones. The bad news is that it requires $\mathcal{O}(n)$ rollouts of the entire policy for n parameters. For a neural net with millions of parameters this is hopeless, and the noise from sampling makes the estimate unreliable.

4.3.2 The score-function (log-likelihood) trick

For differentiable policies there is a much better approach. Use the identity

$$\nabla_{\theta} \pi_{\theta}(a | s) = \pi_{\theta}(a | s) \frac{\nabla_{\theta} \pi_{\theta}(a | s)}{\pi_{\theta}(a | s)} = \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s).$$

The gradient of the log-probability $\nabla_{\theta} \log \pi_{\theta}(a | s)$ is called the **score function**.

Intuition & Mental Model

The score-function identity reads “ $\nabla \pi = \pi \cdot \nabla \log \pi$.” Its purpose is to convert a ∇ that sits *outside* an expectation into a $\nabla \log \pi$ that sits *inside* the expectation, where it can be sampled. This is the same trick that powers REINFORCE, variational inference, and policy gradient methods generally — it is one of the most useful identities in modern ML.

To see why this matters, consider a one-step MDP (a contextual bandit): the agent sees S_t , picks A_t , gets reward R_{t+1} , and the episode ends. The objective is $J(\theta) = \mathbb{E}_{\pi_\theta}[R_{t+1}]$. Naively you might try “sample R and take the gradient,” but R depends on θ only through the action the policy happened to choose — once sampled, R is just a number with no θ inside it.

Apply the trick to push the gradient inside the expectation:

$$\begin{aligned}\nabla_\theta J(\theta) &= \nabla_\theta \sum_s d(s) \sum_a \pi_\theta(a | s) r_{s,a} \\ &= \sum_s d(s) \sum_a r_{s,a} \nabla_\theta \pi_\theta(a | s) \\ &= \sum_s d(s) \sum_a r_{s,a} \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s) \\ &= \mathbb{E}_{\pi_\theta}[R_{t+1} \nabla_\theta \log \pi_\theta(A_t | S_t)].\end{aligned}$$

Now the gradient is an expectation we can sample. The stochastic update is

$$\theta \leftarrow \theta + \alpha R_{t+1} \nabla_\theta \log \pi_\theta(A_t | S_t).$$

This is the **REINFORCE update for one-step MDPs**.

4.3.3 Example: softmax policy

The standard discrete-action policy class is the softmax:

$$\pi_\theta(a | s) = \frac{e^{h_\theta(s,a)}}{\sum_b e^{h_\theta(s,b)}},$$

where $h_\theta(s, a)$ is a learnable preference (logit) for action a in state s . The score function is

$$\nabla_\theta \log \pi_\theta(A_t | S_t) = \underbrace{\nabla_\theta h_\theta(S_t, A_t)}_{\text{preference of chosen action}} - \underbrace{\sum_a \pi_\theta(a | S_t) \nabla_\theta h_\theta(S_t, a)}_{\text{expected preference under policy}}.$$

Compare this to the gradient of softmax cross-entropy in supervised learning — exactly the same shape: “increase the preference of the action you took, decrease the average preference.” The reward-weighted version then says: “do this more strongly when the reward was high.”

In the linear case $h_\theta(s, a) = \theta^\top \phi(s, a)$ the score function simplifies to

$$\nabla_\theta \log \pi_\theta(A_t | S_t) = \phi(S_t, A_t) - \mathbb{E}_{\pi_\theta}[\phi(S_t, \cdot)] = \phi(S_t, A_t) - \sum_a \pi_\theta(a | S_t) \phi(S_t, a),$$

i.e. the difference between the chosen action’s features and the policy’s average features.

4.4 The Policy Gradient Theorem

The score-function trick gives us the gradient for one-step problems. The full multi-step case looks harder: an action A_t affects all future rewards through state transitions, so naively you’d need to differentiate through the environment dynamics — which the agent does not know.

The remarkable result of the **policy gradient theorem** is that you don’t have to. The gradient takes essentially the same shape as the bandit case, with R_{t+1} replaced by the action-value $q_{\pi_\theta}(S_t, A_t)$.

Policy Gradient Theorem (Episodic)

For any differentiable policy $\pi_\theta(a | s)$ and any starting-state distribution d_0 , the gradient of $J(\theta) = \mathbb{E}_{d_0, \pi_\theta}[G_0]$ is

$$\nabla_\theta J(\theta) = \mathbb{E}_{d_0, \pi_\theta} \left[\sum_{t=0}^T \gamma^t q_{\pi_\theta}(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

The gradient does *not* require differentiating through the MDP dynamics.

The proof is short enough to spell out — and the structure is worth seeing once, because the same pattern reappears every time you derive a policy gradient.

Step 1: write J as an expectation over trajectories.. Let $\tau = (S_0, A_0, R_1, S_1, A_1, \dots)$ and $G(\tau) = \sum_t \gamma^t R_{t+1}$. Then

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\pi_\theta}[G(\tau)] = \nabla_\theta \sum_{\tau} p_{\pi_\theta}(\tau) G(\tau).$$

Step 2: apply the score-function trick.. Push ∇_θ inside via $\nabla p = p \nabla \log p$:

$$\nabla_\theta J(\theta) = \sum_{\tau} p_{\pi_\theta}(\tau) G(\tau) \nabla_\theta \log p_{\pi_\theta}(\tau) = \mathbb{E}_{\pi_\theta}[G(\tau) \nabla_\theta \log p_{\pi_\theta}(\tau)].$$

Step 3: expand $\log p_{\pi_\theta}(\tau)$.. The trajectory probability factorizes as

$$p_{\pi_\theta}(\tau) = p(S_0) \prod_{t=0}^T \underbrace{p(S_{t+1} | S_t, A_t)}_{\text{environment dynamics}} \underbrace{\pi_\theta(A_t | S_t)}_{\text{policy}}.$$

Take the log and differentiate w.r.t. θ . The initial-state distribution $p(S_0)$ and the transition probabilities $p(S_{t+1} | S_t, A_t)$ **do not depend on θ** , so their gradients vanish:

$$\nabla_\theta \log p_{\pi_\theta}(\tau) = \sum_{t=0}^T \nabla_\theta \log \pi_\theta(A_t | S_t).$$

The key step

This is the moment the unknown environment dynamics drops out of the gradient. We never need a model of $p(s' | s, a)$. The gradient of the policy's log-probability is enough — and we have that, because we wrote down the policy ourselves.

Step 4: causality / clean-up.. Substituting back gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[G(\tau) \sum_t \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

A causality argument tightens this further: rewards R_{k+1} with $k < t$ cannot depend on A_t (the action hasn't happened yet), so in expectation those terms contribute zero. After splitting $G(\tau) = \sum_k \gamma^k R_{k+1}$ and dropping the zero-mean $k < t$ terms, the remaining reward-tail at step t is the discounted return from t onwards — which in expectation equals $\gamma^t q_{\pi_\theta}(S_t, A_t)$. That gives the theorem. ■

What the policy gradient is doing

At every visited (S_t, A_t) , the gradient says: take the score function $\nabla \log \pi_\theta(A_t | S_t)$ (which points in the parameter direction that increases the probability of A_t in S_t), and weight it by how good the action turned out to be ($q_{\pi_\theta}(S_t, A_t)$). Good actions get their probability pushed up; bad actions get pushed down. The amazing part is that we never had to know how the environment works.

4.4.1 From theorem to algorithm: REINFORCE

The theorem tells us *what* the gradient is, but $q_{\pi_\theta}(S_t, A_t)$ is not directly observable. The simplest unbiased estimate is the actually observed return G_t from a sampled episode:

$$q_{\pi_\theta}(S_t, A_t) = \mathbb{E}_{\pi_\theta}[G_t | S_t, A_t] \approx G_t.$$

Substituting gives the **REINFORCE** algorithm (Williams, 1992):

$$\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t).$$

REINFORCE (Monte-Carlo Policy Gradient)

For each episode $S_0, A_0, R_1, \dots, S_T$ generated by π_θ :

1. Compute returns $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T$ for every t .
2. For each t , update $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t)$.

Monte-Carlo because we use the actual sampled return as the target.

REINFORCE is unbiased — in expectation it tracks the true policy gradient — but it is **noisy**. G_t is a sum of many random rewards, so its variance grows with the episode length, and the resulting gradient estimates can swing wildly between updates.

CartPole as a sanity check

The textbook policy-gradient toy: balance a pole on a moving cart by pushing left or right. State: $S_t = (\text{position}, \text{speed}, \text{angle}, \text{angle-speed})$. Action: $A_t \in \{\text{left}, \text{right}\}$. Reward: +1 for every step the pole stays up; episode ends if it falls or after 200 steps. Policy: a two-layer net with 32 hidden tanh units and a softmax over the two actions.

REINFORCE learns to balance, but the training curve is a noisy zigzag — typical high-variance behavior. Adding a baseline (next section) smooths it out dramatically.

4.5 Reducing Variance: Baselines

REINFORCE works, but its variance is brutal. The good news: we can reduce variance dramatically without introducing any bias.

4.5.1 The baseline identity

Pick any function $b(s)$ that depends only on the state, not on the action. Then

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}}[b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t)] &= \mathbb{E} \left[b(S_t) \sum_a \pi_{\theta}(a | S_t) \nabla_{\theta} \log \pi_{\theta}(a | S_t) \right] \\ &= \mathbb{E} \left[b(S_t) \sum_a \nabla_{\theta} \pi_{\theta}(a | S_t) \right] \\ &= \mathbb{E} \left[b(S_t) \nabla_{\theta} \underbrace{\sum_a \pi_{\theta}(a | S_t)}_{=1} \right] = 0.\end{aligned}$$

The expected “baseline term” is exactly zero, regardless of what b is. So we can subtract $b(S_t)$ from G_t inside the REINFORCE update without changing the gradient in expectation:

$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - b(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

The expected gradient is unchanged. The variance, on the other hand, can be enormously reduced if we choose $b(S_t)$ to track $\mathbb{E}[G_t | S_t]$.

4.5.2 Why a state-value baseline?

The natural choice is the state-value function: $b(s) = v_{\mathbf{w}}(s)$, learned alongside the policy. The advantage interpretation makes this concrete:

$$G_t - v_{\mathbf{w}}(S_t) \approx q_{\pi_{\theta}}(S_t, A_t) - v_{\pi_{\theta}}(S_t) = A_{\pi_{\theta}}(S_t, A_t),$$

the **advantage** of action A_t over the policy’s average behavior in S_t .

Intuition & Mental Model

A baseline doesn’t change *which way* the gradient pushes; it changes *how emphatically*. If every action in some state earns reward around +100, plain REINFORCE strongly increases the probability of *all* of them — but only one of them was actually above average. Subtracting the baseline cancels the common offset so only genuine differences in action quality move the policy. The right question shifts from “was the cumulative reward high or low?” to “was this action better or worse than typical?” On CartPole the practical effect is a much cleaner training curve.

In REINFORCE-with-baseline the value function is itself learned by regression on the same returns:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} (G_t - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t).$$

But it is *only* a baseline — it does not provide the learning signal for the policy. The target for θ is still the Monte-Carlo return G_t .

4.6 Actor-Critic Methods

REINFORCE-with-baseline still uses the full Monte-Carlo return as its target. This is unbiased but high-variance, and it requires waiting for the episode to terminate before any update is possible. The natural next step is to *bootstrap* the return — exactly what TD does for value-based methods.

4.6.1 From baseline to critic

Replace G_t with the one-step bootstrapped estimate $G_{t:t+1} = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$ in the policy update:

$$\begin{aligned}\theta &\leftarrow \theta + \alpha_{\theta} \gamma^t (G_{t:t+1} - v_{\mathbf{w}}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \\ &= \theta + \alpha_{\theta} \gamma^t (R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \\ &= \theta + \alpha_{\theta} \gamma^t \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).\end{aligned}$$

The quantity in parentheses is just the TD error $\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$ — which doubles as a one-step advantage estimate. The value function is now doing two jobs at once: providing the baseline *and* bootstrapping the return.

Actor-Critic (One-Step)

Two parameter sets are learned simultaneously:

- the **actor** $\pi_{\theta}(a | s)$ — the parameterized policy.
- the **critic** $v_{\mathbf{w}}(s)$ — a value function used to estimate advantages and to provide the policy’s learning signal.

At each step compute $\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$, then update

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_{\mathbf{w}} \delta_t \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t), \quad \theta \leftarrow \theta + \alpha_{\theta} \gamma^t \delta_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

The big change relative to REINFORCE-with-baseline is that the critic now *provides* the learning signal, not just a variance reduction. Updates can happen online, every step — no need to wait for the episode to finish.

Bias–variance trade-off again

Pure Monte-Carlo (REINFORCE) is unbiased but high-variance. **One-step bootstrap** (actor-critic) is low-variance but biased — $v_{\mathbf{w}}$ is only an approximation, and bootstrapping propagates its errors. You get faster, smoother learning but pay with some asymptotic correctness. This is exactly the same dial as λ in TD(λ) from the value-based chapter; the policy-gradient world has its own version of it (GAE, below).

4.6.2 n-step actor-critic

The natural interpolation between one-step bootstrap and full Monte-Carlo is the n -step return:

$$G_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l R_{t+l+1} + \gamma^n v_{\mathbf{w}}(S_{t+n}).$$

The corresponding n -step advantage is $\hat{\delta}_t^{(n)} = G_t^{(n)} - v_{\mathbf{w}}(S_t)$, and the actor update uses it in place of δ_t :

$$\theta \leftarrow \theta + \alpha_{\theta} \gamma^t \hat{\delta}_t^{(n)} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Larger n uses more of the actual reward signal (lower bias, higher variance); smaller n relies more on the critic (higher bias, lower variance). Picking n becomes a per-environment tuning problem.

4.6.3 Generalized Advantage Estimation (GAE)

Why pick a single n at all? GAE (Schulman et al., 2015) averages over all of them with an exponential weighting.

Let $\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$ be the per-step TD error. The GAE estimator is

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \delta_{t+1} + (\gamma\lambda)^2 \delta_{t+2} + \dots$$

This is exactly an exponentially-weighted combination of the n -step advantages $\hat{\delta}_t^{(n)}$ for all n , with λ controlling how aggressively longer horizons are downweighted.

Intuition & Mental Model

GAE is the policy-gradient analogue of $\text{TD}(\lambda)$: same exponential weighting, same λ knob.

- $\lambda = 0$: pure one-step TD, $\hat{A}_t = \delta_t$. Maximally biased, minimally noisy.
- $\lambda = 1$: full Monte-Carlo advantage. Unbiased, maximally noisy.
- $0 < \lambda < 1$: smooth interpolation; in practice $\lambda \in [0.9, 0.97]$ is typical.

λ and γ play different roles — γ is the agent’s actual discount on future reward, λ is purely an estimator-side bias/variance knob.

GAE will reappear in the next chapter as the standard advantage estimator for modern actor-critic algorithms like A2C and PPO.

4.7 Recap

Policy-based methods learn the policy directly, by gradient ascent on the expected return. Three big ideas make this work.

- The **score-function trick** converts $\nabla \mathbb{E}[\cdot]$ into $\mathbb{E}[\nabla \log \pi \cdot \cdot]$, so the gradient becomes a sample-able expectation.
- The **policy gradient theorem** extends the trick to multi-step MDPs and shows that we never need to differentiate through the environment.
- **Baselines and critics** keep the gradient unbiased (or only mildly biased) while drastically reducing its variance — without them REINFORCE is almost too noisy to be useful.

The next chapter pushes this further with modern actor-critic algorithms: deep, batched, and engineered to actually be stable on hard problems.

Modern Actor-Critic Algorithms

Plain actor-critic with GAE is the algorithmic skeleton. Making it actually work on hard problems — continuous control, sparse rewards, real-world robotics, language-model fine-tuning — requires several more layers. This chapter walks through the modern landscape: NPG, TRPO, PPO on the on-policy side; DDPG, TD3, SAC on the off-policy / continuous-control side.

Three concrete weaknesses of vanilla actor-critic motivate everything below. First, it is **on-policy** — data collected with the current π_θ cannot be reused after θ changes, which is wasteful. Second, its updates are **unstable** — small steps in parameter space can produce huge changes in action probabilities, and one bad step contaminates all future data. Third, even with baselines and GAE the gradient estimates remain **noisy**. Modern algorithms attack each of these weaknesses with a different tool.

5.1 A Unified View of Policy Gradients

Every policy-gradient method we have seen — and most we are about to see — shares the same structural form:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \Psi_t \nabla_\theta \log \pi_\theta(A_t | S_t) \right],$$

where Ψ_t is the *learning signal* for the action taken at time t . Different algorithms differ only in the choice of Ψ_t :

Choice of Ψ_t	Algorithm
G_t (Monte-Carlo return)	REINFORCE
$G_t - v_{\mathbf{w}}(S_t)$	REINFORCE with baseline
$\delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)$	TD(0) actor-critic / A2C
$G_t^{(n)} - v_{\mathbf{w}}(S_t)$	n -step actor-critic
$\sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$	GAE (used in PPO)
$q_{\mathbf{w}}(S_t, A_t)$	Q-based actor-critic (DDPG, SAC)

Reading top-to-bottom, this is a spectrum from **maximum-bias-zero** / **variance-high** (REINFORCE) to **low-bias** / **low-variance** (actor-critic with a learned critic). The job of each algorithm is to pick a point on this spectrum and pair it with a way of stabilising the resulting updates.

5.1.1 Bias and variance, again

Every Ψ_t sitting below the first row uses a learned value function and is therefore *biased* — the critic is only an approximation, and bootstrapping its errors propagate.

Compatible function approximation (briefly)

There *is* a way to pair the critic with the actor so that the policy-gradient estimate stays unbiased: choose features for the critic that are “compatible” with the policy’s score function (Sutton et al., 2000). The compatible features are $\nabla_{\theta} \log \pi_{\theta}(s, a)$ themselves, and the critic is restricted to a linear function of those features. This is mathematically beautiful but in deep RL almost nobody uses it: the linearity constraint is way too restrictive for neural networks. The pragmatic deep-RL stance is “low variance plus an expressive critic beats unbiasedness.”

5.1.2 Why γ^t usually disappears

Looking back at the policy gradient theorem, every term carried a γ^t in front of the score function. In the slides and in most practical code that γ^t is silently dropped. The reason is pragmatic: with γ^t , late-step gradients are exponentially down-weighted, and most of the learning signal comes from the first few steps of each episode — which works poorly in practice. Dropping it gives a (slightly) biased estimator of the discounted return objective but a much better one for the average-reward objective, and it is what every modern implementation actually does. We follow that convention from now on.

5.2 Off-Policy Policy Gradients

The on-policy nature of REINFORCE / actor-critic is a real annoyance. Every gradient step requires fresh trajectories from the current π_{θ} , and as soon as we update, all the previously collected data is technically off-policy. Throwing away data after one update is wasteful — can we keep using it?

5.2.1 Importance sampling

If data was collected by some *behavior policy* $\mu(a | s)$ but we want expectations under the *target policy* $\pi(a | s)$, the standard fix is importance sampling:

$$\mathbb{E}_{a \sim \pi}[f(s, a)] = \mathbb{E}_{a \sim \mu} \left[\frac{\pi(a | s)}{\mu(a | s)} f(s, a) \right].$$

The ratio $w = \pi/\mu$ is the per-action **importance weight**; large w means the target policy would have taken this action much more often than the behavior policy did, and we re-weight accordingly.

Applied to the policy gradient, the off-policy version is

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\mu} \left[\frac{\pi_{\theta}(a | s)}{\mu(a | s)} \nabla_{\theta} \log \pi_{\theta}(a | s) G_t \right],$$

or, if we want to reweight an entire trajectory τ collected under μ ,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \mu} \left[\prod_{t=0}^T \frac{\pi_{\theta}(a_t | s_t)}{\mu(a_t | s_t)} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G(\tau) \right].$$

Why per-trajectory IS is doomed

A product of T ratios is a recipe for variance disaster. If even one ratio is 0.1 or 10, the whole trajectory’s contribution swings by an order of magnitude. Over hundreds of timesteps the products either explode or collapse to zero, and the gradient estimate becomes useless. **Per-decision** importance sampling — correcting only up to time t , not the whole future — helps somewhat, but the underlying problem (unbounded variance from ratio products) remains.

In practice, deep-RL methods deal with this in one of two ways. **Constrain** the policy to stay close to the data-collecting policy, so the ratios stay bounded (TRPO, PPO — next section). Or **avoid** importance sampling entirely and rely on a replay buffer + bootstrapped TD targets, which sidesteps the distribution-mismatch problem at the cost of additional bias (DDPG, SAC — continuous-control section).

5.3 Stable Updates: NPG, TRPO, PPO

Even on-policy, large updates are dangerous. A small step in θ can move action probabilities a lot — especially when the policy is near-deterministic and the score function blows up. A bad update produces a worse policy, which produces worse data, which produces an even worse update. Easy to spiral.

The cure is to control updates in *policy space*, not parameter space.

5.3.1 Measuring policy change with KL divergence

The natural way to measure “how different are these two policies?” is the KL divergence over action distributions:

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) = \mathbb{E}_{s \sim d_{\pi_{\theta_{\text{old}}}}} \left[\sum_a \pi_{\theta_{\text{old}}}(a \mid s) \log \frac{\pi_{\theta_{\text{old}}}(a \mid s)}{\pi_{\theta}(a \mid s)} \right].$$

Small KL means similar action distributions in the states the old policy actually visits. The constrained update is then

$$\max_{\theta} J(\theta) \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \leq \epsilon.$$

The trust region ϵ caps how far the policy can drift in one update — which keeps both the gradient estimates and the importance ratios well-behaved.

5.3.2 Natural Policy Gradient (NPG)

The plain gradient $\nabla_{\theta} J$ is the steepest direction in *parameter* space, not in policy space. The Natural Policy Gradient (Kakade, 2002) rescales it by the inverse of the local curvature of KL:

$$\theta \leftarrow \theta + \alpha F^{-1} \nabla_{\theta} J(\theta),$$

where F is the **Fisher information matrix**, $F = \mathbb{E}[\nabla \log \pi_{\theta} \nabla \log \pi_{\theta}^{\text{T}}]$, the local Hessian of the KL in θ .

Intuition & Mental Model

You can think of NPG as gradient ascent in a coordinate system where unit distance corresponds to a unit of KL divergence. A step of fixed length always changes the policy by roughly the same amount, regardless of how the parameters happen to be scaled. The price is the Fisher matrix, which is expensive to invert for neural networks (usually approximated with conjugate gradient). And NPG only uses a *local* approximation to KL — nothing prevents a single large step from violating the trust region.

5.3.3 Trust Region Policy Optimization (TRPO)

TRPO (Schulman et al., 2015) makes the KL constraint explicit, optimizing a **surrogate objective** that’s equivalent to $J(\theta)$ for small KL:

$$\max_{\theta} \mathbb{E}_{(s,a) \sim \pi_{\theta_{\text{old}}}} \left[\underbrace{\frac{\pi_{\theta}(a | s)}{\pi_{\theta_{\text{old}}}(a | s)}}_{r_t(\theta)} \hat{A}_t \right] \quad \text{s.t.} \quad D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_{\theta}) \leq \epsilon.$$

Three things are happening here. The expectation is over data collected with $\pi_{\theta_{\text{old}}}$ (so we can reuse recent rollouts, not just the latest one). The importance ratio $r_t(\theta)$ corrects for the distribution mismatch. The KL constraint keeps the ratio close to 1, which keeps the surrogate a faithful local approximation of the true objective.

Why a surrogate objective?

The true objective $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)]$ has two practical problems:

- We can’t evaluate it without fresh data from π_{θ} .
- The data we have was collected under a different policy.

The surrogate solves both: it uses old data (plus a ratio correction) and it’s *provably* a local approximation to J when KL stays small. Solve the surrogate with the trust region; you’ve effectively done a controlled step on the true objective.

TRPO is robust and sample-efficient, but algorithmically heavy — second-order optimization, conjugate gradient, and a line search to enforce the KL bound.

5.3.4 Proximal Policy Optimization (PPO)

PPO (Schulman et al., 2017) replaces TRPO’s hard KL constraint with a much simpler trick: **clip the ratio** so the optimization can never benefit from a too-large update:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_{(s,a) \sim \pi_{\theta_{\text{old}}}} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

with $r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ and ϵ typically around 0.1–0.2.

PPO’s clipped objective in plain English

For a sample with positive advantage ($\hat{A}_t > 0$, “this action was better than average”):

- If $r_t(\theta) \leq 1 + \epsilon$, the gradient pushes up the action’s probability normally.

- If $r_t(\theta) > 1 + \varepsilon$, the clip kicks in and the gradient becomes zero — you’ve already increased the probability enough; stop.

Symmetrically for $\hat{A}_t < 0$: probability is decreased until $r_t(\theta) = 1 - \varepsilon$, then no further. The minimum with the unclipped term ensures the optimizer cannot exploit the clip to undo a prior update.

Intuition & Mental Model

PPO is essentially TRPO with all the heavy machinery thrown away. No Fisher matrix, no conjugate gradient, no line search — just first-order SGD on a clipped objective. The trust-region effect now comes from the clip itself: the surrogate stops changing once the ratio leaves $[1 - \varepsilon, 1 + \varepsilon]$, so further gradient steps in that direction do nothing useful. The result is that PPO can do *multiple epochs of updates over the same batch* of trajectories — vanilla actor-critic can only do one before the data goes stale. That alone is a huge win in sample efficiency.

PPO uses GAE for the advantage estimate, often adds an entropy bonus to keep the policy from collapsing prematurely, and is the workhorse for large-scale on-policy deep RL: Atari, MuJoCo, robotics simulators, RLHF for language models.

5.3.5 NPG, TRPO, PPO at a glance

Method	Constraint	Cost	Key idea
NPG	implicit KL (Fisher)	second-order	natural gradient / KL geometry
TRPO	hard KL bound	second-order	explicit trust region
PPO	soft via clipping	first-order	cheap clipped surrogate

All three are on-policy in the sense that data must come from $\pi_{\theta_{\text{old}}}$, but the trust region lets us *reuse* that data for several update steps. The trade-off across the three is essentially complexity vs. rigor: NPG/TRPO have cleaner theory; PPO is what people actually run.

5.4 Continuous Control

Everything so far has been algorithm-agnostic about action spaces. Continuous actions just require a different policy parameterization — the same gradient machinery still works.

5.4.1 The Gaussian policy

The standard stochastic policy for continuous control is a diagonal Gaussian:

$$A_t \sim \pi_{\theta}(\cdot | S_t) = \mathcal{N}(\mu_{\theta}(S_t), \Sigma_{\theta}(S_t)),$$

where the network outputs the mean $\mu_{\theta}(s)$ and (optionally) a state-dependent covariance $\Sigma_{\theta}(s)$. Often the covariance is a single learned vector independent of s . The score function for fixed scalar variance σ^2 is

$$\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) = \frac{A_t - \mu_{\theta}(S_t)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(S_t).$$

Read this as: *nudge the mean toward the chosen action A_t , weighted by how unusual the action was given the current mean*. Combined with a positive advantage, the policy update moves the

mean toward high-advantage samples; combined with a negative advantage, it moves the mean away from them.

5.4.2 Deterministic Policy Gradient (DPG)

For continuous actions there is also a non-stochastic alternative. If the critic $q_{\mathbf{w}}(s, a)$ is differentiable in a , we can take gradients *through* the critic to improve a deterministic policy $a = \mu_{\theta}(s)$:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d_{\mu}} \left[\nabla_a q_{\mathbf{w}}(s, a) \big|_{a=\mu_{\theta}(s)} \nabla_{\theta} \mu_{\theta}(s) \right].$$

Intuition & Mental Model

The deterministic policy gradient avoids the high-variance “sample then weight by score function” loop. Instead, it asks the critic directly: “which way in action space increases value?” and adjusts the actor’s mean accordingly. The price: we need a critic that is accurate *and smooth* in a . If the critic has sharp peaks or is wrong about the action gradient in some region, the actor chases bad signals — which is why DPG-family methods need extra stabilization.

5.4.3 Deep Deterministic Policy Gradient (DDPG)

DDPG (Lillicrap et al., 2015) is essentially *DQN for continuous actions*: DPG, plus a replay buffer for off-policy learning, plus target networks for stability.

- **Critic update** (TD regression on tuples sampled from the buffer):

$$\min_{\mathbf{w}} \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[(q_{\mathbf{w}}(s, a) - y)^2 \right], \quad y = r + \gamma q_{\mathbf{w}^-}(s', \mu_{\theta^-}(s')).$$

- **Actor update** (deterministic policy gradient through the critic):

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [q_{\mathbf{w}}(s, \mu_{\theta}(s))].$$

- Target networks $\mathbf{w}^-, \theta^-$ track the live ones with a slow Polyak average $\mathbf{w}^- \leftarrow \tau \mathbf{w} + (1 - \tau) \mathbf{w}^-$.
- Exploration is added by hand: $a_t = \mu_{\theta}(s_t) + \mathcal{N}_t$ with some Gaussian or Ornstein–Uhlenbeck noise.

DQN ↔ DDPG correspondence

Both algorithms are off-policy + replay + target nets + bootstrapped TD targets, with the same critic structure $q(s, a)$. The only difference is how the greedy action is obtained: DQN takes $\arg \max_a q(s, a)$ explicitly (cheap for discrete actions); DDPG learns a parameterized maximizer $\mu_{\theta}(s) \approx \arg \max_a q_{\mathbf{w}}(s, a)$ (necessary for continuous actions). DDPG = DQN + a learned argmax.

DDPG works, but it is brittle. The critic tends to *overestimate* Q-values, and those errors feed straight into the actor.

5.4.4 TD3

TD3 (Fujimoto et al., 2018) is DDPG with three concrete fixes for that brittleness:

- **Clipped double-Q learning:** keep two critics $q_{\mathbf{w}_1}, q_{\mathbf{w}_2}$ and use $\min(q_1, q_2)$ in the target. Two independent overestimates rarely peak at the same action, so taking the minimum suppresses the bias.
- **Delayed policy updates:** update the critic on every step but the actor only every d steps. Lets the critic catch up before chasing it.
- **Target policy smoothing:** add a small clipped Gaussian noise to the target action $\mu_{\theta}(s') + \tilde{\epsilon}$ in the critic target. Prevents the actor from exploiting sharp peaks in the (still imperfect) critic.

The result is the standard “clean” off-policy continuous-control baseline — TD3 on MuJoCo benchmarks is what every new paper is compared against.

5.4.5 Maximum-entropy RL and SAC

DDPG and TD3 still use *deterministic* policies, with exploration tacked on as external action noise. That noise is brittle (state-independent, often poorly tuned) and the policy can collapse to a single suboptimal mode early in training.

The cleaner approach is to learn a **stochastic** policy whose stochasticity is itself part of the objective. **Maximum-entropy RL** adds an entropy bonus:

$$J_{\text{ent}}(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t (R_{t+1} + \alpha H(\pi_{\theta}(\cdot | S_t))) \right],$$

where $H(\pi(\cdot | s)) = -\mathbb{E}_{a \sim \pi}[\log \pi(a | s)]$ is the entropy of the action distribution and $\alpha > 0$ is a temperature.

Why entropy as objective, not regularizer?

Adding entropy to the objective changes *which policy is optimal*, not just how we search for it. The optimal policy for the augmented objective has high reward *and* high stochasticity. This means: sustained exploration (the policy refuses to become deterministic too early), robustness to value errors (a stochastic policy hedges across action choices), and smoother updates. A small entropy bonus is sometimes added to PPO too, but in SAC it is the principled core of the method.

Soft Actor-Critic (SAC) (Haarnoja et al., 2018) puts the maximum-entropy objective together with the off-policy machinery of DDPG/TD3:

- Replay buffer (off-policy).
- Two critics with target networks, trained with the entropy-augmented TD target

$$y = r + \gamma \mathbb{E}_{a' \sim \pi_{\theta}} [q_{\mathbf{w}}(s', a') - \alpha \log \pi_{\theta}(a' | s')].$$

- Stochastic Gaussian (typically squashed-Gaussian) policy, updated to maximize $\mathbb{E}_{a \sim \pi_{\theta}} [q_{\mathbf{w}}(s, a) - \alpha \log \pi_{\theta}(a | s)]$ via the reparameterization trick.
- Optionally, learn α itself by gradient descent against a target entropy.

5.4.6 When to use what

Modern deep RL has roughly three workhorse algorithms, each with a distinct sweet spot:

	PPO	TD3	SAC
Policy	stochastic	deterministic	stochastic
Data regime	on-policy	off-policy	off-policy
Best fit	stable large-scale training	clean continuous control baseline	robust, sample-efficient control
Use cases	simulators, LLMs, games (RLHF)	benchmark control, precise actuation	robotics, real-world control
Main trade-off	data hungry	weaker exploration	more moving parts

Intuition & Mental Model

Choosing between PPO, TD3, and SAC is a *systems decision*, not just an algorithmic one. **PPO**: when you can run thousands of parallel environments cheaply and want something simple that almost always works. **TD3**: when you want the cleanest baseline for continuous benchmarks. **SAC**: when each environment step is expensive (real robots) and you need sample efficiency plus robust exploration.

5.5 Recap

The progression from REINFORCE to modern actor-critic is a story of **controlled trade-offs**.

- All policy-gradient methods share the form $\nabla J = \mathbb{E}[\Psi_t \nabla \log \pi]$. Different choices of Ψ_t trade bias for variance.
- Vanilla actor-critic has three weaknesses — on-policy, unstable, noisy — and modern algorithms each address them differently.
- **Trust-region methods (TRPO/PPO)** stabilize on-policy updates by constraining the policy in KL space. PPO is what people actually run.
- **Off-policy actor-critics (DDPG/TD3/SAC)** reuse data via a replay buffer, use a parameterized maximizer for continuous actions, and stabilize the critic with target networks, double-Q learning, and (in SAC) maximum-entropy policies.

The next chapter zooms out: even with the best of these algorithms, deep RL still fails on many problems. The bottlenecks are usually not algorithmic — they’re about exploration, credit assignment, and state representation.

Core Challenges in Deep RL

By now we have an algorithmic toolbox: DQN and its variants for value-based discrete control, PPO for stable on-policy optimization, DDPG / TD3 / SAC for off-policy continuous control. So why does deep RL still fail on so many real problems?

The honest answer is that the algorithms are usually not the bottleneck. The same PPO that solves a benchmark in a few hours can fail completely on a problem that *looks* similar but has a sparser reward, a higher-dimensional observation, or a longer horizon. Performance depends on the environment, the reward design, and the state representation almost as much as on the algorithm itself.

Three core challenges sit underneath most failure modes:

- **Exploration** — collecting data that is informative and reward-relevant, not just data.
- **Credit assignment** — figuring out which past actions caused the current outcome.
- **State representation** — learning compact, informative latent states from high-dimensional, possibly partial observations.

These are deeply interconnected: better representations help both exploration and credit assignment; better exploration provides the data needed for accurate credit signals; accurate credit assignment guides the representation toward decision-relevant features.

This chapter walks through each in turn.

6.1 Exploration

Exploration in RL means collecting data that is *informative* and *reward-relevant*. Too little exploration and the agent locks in on a bad policy; too much random exploration and it wastes samples on irrelevant states. The goal isn't "more randomness" — it is exploring the right states at the right time.

6.1.1 Why exploration is hard

Three structural features make exploration genuinely difficult in deep RL:

- **Sparse rewards.** Useful feedback is rare. The agent could stumble around for thousands of steps without seeing a single non-zero reward.
- **High-dimensional state and action spaces.** The combinatorial explosion of possibilities makes random exploration vanishingly unlikely to hit anything good.

- **Long horizons.** A correct decision early on may pay off only much later — and the agent has to keep making the right calls in between.

Montezuma’s Revenge

The classic poster child for hard exploration. The agent navigates an Atari maze of rooms, ladders, traps, and keys. Reward is given only after long, precise sequences of actions (pick up a key, descend, dodge skulls, climb back up, open a door). For years, plain DQN scored *zero* — the policy never reached the first reward and so had nothing to learn from. Algorithms only made progress by adding directed, exploration-aware bonuses.

6.1.2 Baseline exploration: random and noise-based

The exploration tools we have already met are basic and undirected.

- **ε -greedy** (value-based RL): act greedily with probability $1 - \varepsilon$, else uniform random. Simple, widely used, fine in small or dense-reward problems — hopeless when good behavior is more than a few random actions away.
- **Action-space noise** (DDPG/TD3): $a_t = \mu_\theta(s_t) + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Easy to add for continuous actions but the noise is independent across time and state, leading to jittery, undirected exploration.
- **Entropy regularization** (PPO/SAC): add an entropy bonus to the objective so the policy doesn’t collapse too soon. Preserves stochasticity, but “stochastic” isn’t the same as “directed.”
- **Noisy networks** (Rainbow): replace deterministic linear layers with stochastic ones whose noise scales are *learned*, giving state-dependent exploration. A clear improvement over ε -greedy — but still implicit, no explicit notion of what is unknown.

What’s missing from these methods

All of the above are essentially answers to “how should I randomize my actions?” None of them answer the actually important question: *where should I explore next?* That question requires a notion of **uncertainty**, **novelty**, or **surprise** — something that distinguishes “state I have already seen plenty of” from “state I know nothing about.”

6.1.3 Three guiding principles for directed exploration

The literature on directed exploration revolves around three ideas:

- **Uncertainty:** explore where the value estimate is uncertain.
- **Novelty:** explore states the agent has rarely visited.
- **Surprise:** explore transitions that the agent’s model fails to predict.

These overlap in practice and most modern methods combine more than one.

6.1.4 Uncertainty-based exploration

The principled view: in early training the agent has **epistemic uncertainty** about the true value function (the kind of uncertainty that shrinks as you collect more data). Exploration should target state-action pairs where that uncertainty is high.

The bandit literature gives two canonical strategies that scale to deep RL:

- **UCB (optimism in the face of uncertainty):** act on the upper confidence bound, $a_t = \arg \max_a [\mu_t(a) + \beta \sigma_t(a)]$. Combines the estimated value with a bonus for high uncertainty.
- **Thompson sampling:** sample a value-function hypothesis from the posterior, act greedily under that sample. Probabilistic and naturally exploration–exploitation balanced.

Both require a way to estimate uncertainty over Q -values. Deep RL has no tractable posterior over neural-network parameters, so we approximate.

Ensembles.. Maintain K different Q -networks $Q^{(1)}, \dots, Q^{(K)}$ trained on the same data. The variance across the ensemble at (s, a) is a proxy for epistemic uncertainty. Use it for UCB-style action selection or Thompson-style sampling of an acting head.

Bootstrapped DQN.. Train each ensemble member on a different bootstrap subsample of the replay buffer (via per-transition masks). Sample one acting head per episode and act greedily under it for the whole episode. This produces **deep exploration**: temporally consistent, multi-step exploration, rather than the per-step noise of ε -greedy.

Randomized priors.. A subtle failure mode of ensembles: in regions with little data, all heads can drift to the same uninformative value, becoming overconfidently identical. The fix is to add a *fixed random prior function* $p^{(j)}(s, a)$ to each head: $\hat{Q}^{(j)}(s, a) = Q^{(j)}(s, a) + \lambda p^{(j)}(s, a)$. Train only $Q^{(j)}$; keep $p^{(j)}$ fixed. The frozen priors guarantee head diversity in unvisited regions, sustaining exploration even under sparse rewards.

6.1.5 Novelty-based exploration: intrinsic motivation

A different strategy: shape the reward so that visiting novel states is intrinsically rewarding. Combine the environment reward with an **intrinsic** bonus:

$$r_t = r_t^{\text{ext}} + \beta r_t^{\text{int}},$$

where r_t^{int} measures how new or surprising the current transition is.

Count-based exploration.. The clean tabular version: $r_t^{\text{int}}(s_t) = 1/\sqrt{N(s_t)}$ where $N(s_t)$ is the visitation count. Rare states get a big bonus that decays as they are visited more. In deep RL exact counts are impossible (no two pixel observations are ever identical), so we approximate via **pseudo-counts** from a density model, or via state hashing to map continuous observations to discrete bins.

Curiosity / prediction error.. Instead of counting, measure how much the agent’s model *fails* to predict. **Random Network Distillation (RND)** fixes a random target network f_{target} and trains a predictor f_θ to mimic it on visited states. The intrinsic reward is the prediction error $r_t^{\text{int}} = \|f_\theta(s_t) - f_{\text{target}}(s_t)\|^2$. Unseen states give high error; familiar states have low error. Simple, no environment model required, and remarkably effective on hard-exploration Atari games.

The **Intrinsic Curiosity Module (ICM)** learns a feature encoder $\phi(s)$ together with a forward model $\hat{\phi}(s_{t+1}) = f(\phi(s_t), a_t)$ and rewards prediction error in feature space, $r_t^{\text{int}} = \|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|^2$.

Disagreement-based.. Train an ensemble of forward models, use the variance across their predictions as the intrinsic reward. High disagreement = uncertain dynamics = worth exploring. This isolates *epistemic* uncertainty (which more data can fix) from *aleatoric* noise (which it cannot).

Combining signals (NGU).. Modern methods combine multiple novelty signals. **Never Give Up** (Badia et al., 2020) uses both *episodic novelty* (how often the state was visited within the current episode) and *lifelong novelty* (RND-style: how unfamiliar across all training so far). The episodic component encourages within-episode exploration; the lifelong component maintains long-term exploration pressure.

The noisy-TV problem

Curiosity-style intrinsic rewards have a famous failure mode. Imagine an Atari-like environment containing a TV that displays random noise. The prediction error of any model on the TV’s pixels is permanently high — the noise is unpredictable by definition. A naive curiosity-driven agent will sit in front of the TV forever, collecting maximum intrinsic reward and learning nothing useful. The fundamental issue: prediction error doesn’t distinguish “unfamiliar but learnable” from “random and unlearnable.” Practical mitigations: normalize/clip intrinsic rewards, anneal β over training, and combine multiple novelty signals (e.g. NGU’s episodic + lifelong).

6.1.6 Where exploration fits in the pipeline

Exploration can be injected at three different levels of the RL pipeline:

- **Action selection:** ϵ -greedy, UCB, Thompson sampling, action noise.
- **Model / value estimation:** noisy networks, ensembles, randomized priors.
- **Reward shaping:** count-based bonuses, RND, ICM, disagreement.

The right level depends on the problem. For dense-reward control, action noise is usually enough. For sparse-reward, long-horizon, or partially-observable problems, you almost always need something at the value-estimation or reward-shaping level.

6.2 Credit Assignment

The **credit assignment problem**: when an outcome arrives, which past actions caused it? This is hard because most real problems have delayed and sparse feedback, with many irrelevant decisions sandwiched in between. The agent needs to disentangle informed choices from serendipitous outcomes — and credit assignment is what does that.

Credit vs. value

Value asks “what return do I expect from this state?” Credit asks “how much did this specific action contribute to that outcome?” Two related but distinct questions. A good value estimate doesn’t automatically give good credit signals — if the value drifts equally with both helpful and irrelevant actions, every action gets the same credit.

6.2.1 Three root causes

Hard credit assignment usually comes from one of three structural features of the MDP:

- **Depth** — the temporal distance between an action and its outcome is large. TD errors shrink exponentially as they propagate backward, so important actions far from the reward get tiny gradients. Short-sighted methods develop a *recency bias*, attributing credit to whichever action happened to be near the reward.
- **Density** — few of the actions actually influence the outcome (sparse-reward setting). Rare goal states mean most TD errors carry no information, so updates are dominated by zero-information samples.
- **Breadth** — many alternative paths lead to the same outcome. Credit gets diluted across all the equivalent trajectories, with no clear bottleneck action to learn.

Knowing which of the three is dominant tells you which technique to reach for. No single method solves all three.

6.2.2 Time-contiguity methods (depth)

The default toolbox for moderate delays:

- **TD(1)**: reward propagates backward one step per update. Slow on long horizons.
- **n -step returns**: reward propagates back n steps in one shot. Faster, but higher variance.
- **Eligibility traces / TD(λ)**: exponentially-weighted combination of all n -step returns. The same idea on the policy-gradient side is **GAE** from Chapter 4, which is now the default advantage estimator.

Larger n or larger λ accelerates credit propagation at the cost of variance. These methods address depth directly — but if the depth is hundreds of steps and the reward is sparse, even n -step methods struggle.

6.2.3 Reducing breadth noise with advantages

Many actions in a trajectory simply don't matter — they don't change the outcome. Naively, all of them get blamed/credited for the eventual reward, contributing noise. The advantage function $A(s, a) = q(s, a) - v(s)$ already does the right thing here: only *above-* or *below-average* actions get nonzero credit, irrelevant ones average out. TD handles *when* reward is propagated; advantage handles *whether the action mattered*. They are complementary, which is why GAE-style advantage estimation has become standard.

6.2.4 Return decomposition (RUDDER) for very long delays

When the reward is genuinely far from its cause, even n -step / λ -returns are not enough. **RUDDER** (Arjona-Medina et al., 2019) takes a different approach: *move the reward to the important event*.

The idea: train a sequence model (e.g. an LSTM) to predict the trajectory's future return $\hat{G}_t$ from $(s_0, a_0, \dots, s_t)$. Then redistribute reward so that each step gets the *change* in predicted return:

$$\bar{r}_{t+1} = \hat{G}_{t+1} - \hat{G}_t.$$

A large jump in predicted return marks an important event (“ah, this is when I realized I'd win”). After redistribution, the long-horizon, sparse-reward problem becomes a much easier local-credit problem, because reward now arrives at the actual causal moment.

6.2.5 Hindsight Experience Replay (HER) for sparse goals

For **goal-conditioned** problems with binary reward (you reached the goal or you didn't), most early trajectories fail and provide no learning signal — the density challenge in pure form.

Hindsight Experience Replay (Andrychowicz et al., 2017) flips the question: every failed trajectory *succeeded at some other goal*, namely the state it actually reached. Relabel transitions as

$$(s_t, a_t, r_t, g) \longrightarrow (s_t, a_t, r'_t, g'),$$

where $g' = s_T$ (or some achieved state) and $r'_t = 1$ iff $s_t = g'$. Train on both the original and the relabeled experience.

Intuition & Mental Model

HER turns the sparse failure signal into dense training data: even when the agent never reaches the original goal, it can still learn a useful goal-conditioned policy by reaching whatever it actually reached. Over many episodes the agent learns to reach a wide variety of subgoals — and as that capability accumulates, reaching the intended goal becomes feasible. The catch: the MDP must be *goal-conditioned*, with reward structure that generalizes across goals. Not every problem has that shape.

6.2.6 Practical takeaways

- **Mild delay:** TD, n -step, GAE. The default tools, almost always present.
- **Long delay:** return decomposition (RUDDER) when delays are extreme and causal events are sparse.
- **Sparse goal-conditioned reward:** hindsight relabeling (HER).
- Diagnose first: depth vs. density vs. breadth. The right method depends on which one is binding.

Credit assignment is also tightly coupled to representation: bad features make accurate credit assignment impossible, no matter the algorithm.

6.3 State Representation Learning

Raw observations are usually high-dimensional (image pixels, lidar scans), noisy, and often partial. A good **state representation** compresses these into a latent vector that keeps the control-relevant information and discards the rest.

6.3.1 End-to-end RL vs. SRL + RL

End-to-end.. Train a single policy/value network straight from observations to actions. The early layers implicitly learn features. This is the default in Atari-style benchmarks. The downsides: features are entirely shaped by the (often weak) RL gradient, generalization across tasks is poor, and sample efficiency is low.

SRL + RL.. Decouple the encoder from the policy. First (or jointly) train an encoder $\phi : o \rightarrow x$ that produces a useful latent state, then run RL on top of x . Modular, often more sample-efficient, and the encoder transfers across tasks.

The trade-off is essentially *generality of the representation vs. alignment with the control objective*. Pure end-to-end is maximally aligned but data-hungry; pure SRL is maximally general but the auxiliary objective may not match what the policy needs.

6.3.2 What makes a representation good

Three desiderata are usually invoked:

- **Structure:** the latent space should reflect the causal, task-relevant structure of the environment, not arbitrary visual variation.
- **Continuity:** nearby states in the world should map to nearby latents. Small observation changes should produce small representation changes.
- **Sparsity / disentanglement:** only a small number of latent dimensions should be active per state. Reduces spurious correlations and makes downstream learning easier.

6.3.3 A taxonomy of SRL methods

The course groups online model-free SRL into six (overlapping) classes.

Metric-based / bisimulation.. Shape the latent metric so that two states are close iff they produce similar rewards and similar transition dynamics. Bisimulation is the principled formalization (Ferns et al., 2004): states are *bisimilar* iff for every action they yield the same reward and transition to bisimilar states. Strong abstraction, expensive to compute.

Auxiliary tasks.. Add side losses to the encoder: *reconstruction* (recover the observation from the latent), *forward dynamics* (predict $\phi(s_{t+1})$ from $\phi(s_t)$ and a_t), *inverse dynamics* (predict a_t from $\phi(s_t)$ and $\phi(s_{t+1})$), *reward prediction*. Provides a richer signal than the RL gradient alone, especially under sparse reward. The risk: an auxiliary loss may not align with what the controller needs (reconstruction wastes capacity on visually salient but task-irrelevant pixels).

Data augmentation.. Apply visual augmentations (random crops, color jitter, masking, noise injection) to observations and require the encoder to be invariant. Cheap, and it consistently improves both sample efficiency and generalization in pixel-based RL. The caveat: too strong an augmentation can destroy task-relevant cues.

Contrastive SRL.. Train the encoder so that positive pairs (same state under different augmentations, or temporally close observations) are close in latent space and negative pairs (random other states) are far. The standard NCE-style loss

$$\mathcal{L}_{\text{NCE}} = -\mathbb{E} \left[\log \frac{\exp(\text{sim}(\phi(o), \phi(o^+)))}{\exp(\text{sim}(\phi(o), \phi(o^+))) + \sum_i \exp(\text{sim}(\phi(o), \phi(o_i^-)))} \right]$$

encourages a clustered, well-structured latent space without needing reconstructions or labels.

Non-contrastive.. Same goal as contrastive (similar states close in latent space) but without explicit negative pairs — using e.g. predictor networks and target encoders to prevent representational collapse (BYOL-style methods).

Attention-based.. Learn attention masks over spatial or temporal features so that the encoder focuses on decision-relevant regions and suppresses distractors. Improves interpretability and robustness; adds architectural complexity.

6.3.4 Why representation matters in practice

A weak representation makes everything downstream harder. Bad features mean noisy value estimates, which produce noisy gradients, which produce noisy policies — and the same algorithm that succeeds on a clean benchmark fails on a visually messy real-world task. Most progress in pixel-based deep RL over the last few years has come from better encoders (data augmentation, contrastive losses) rather than better RL algorithms.

6.4 Recap

Modern deep RL is bottlenecked by three intertwined problems, none of which is solved by “pick a better algorithm.”

- **Exploration:** how to discover rewarding behavior in large, sparse, or deceptive environments. Tools: noisy networks, uncertainty ensembles, bootstrap with randomized priors, intrinsic motivation (RND, ICM, disagreement, NGU).
- **Credit assignment:** how to attribute outcomes to past actions across long, sparse, or branching horizons. Tools: TD / n -step / GAE for moderate delays, advantages for breadth, RUDDER for long delays, HER for sparse goal-conditioned reward.
- **State representation:** how to learn compact, informative latents from high-dimensional observations. Tools: bisimulation, auxiliary tasks, data augmentation, contrastive and non-contrastive SRL, attention.

These three challenges feed into one another. Better representations make exploration and credit assignment tractable. Better exploration provides the data needed for learning representations and assigning credit. Better credit assignment guides the representation toward features that actually matter for control. Real-world deep RL is usually about getting all three right simultaneously — not finding a single magic algorithm.